Foλang Programming Language

Foλang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Design Overview

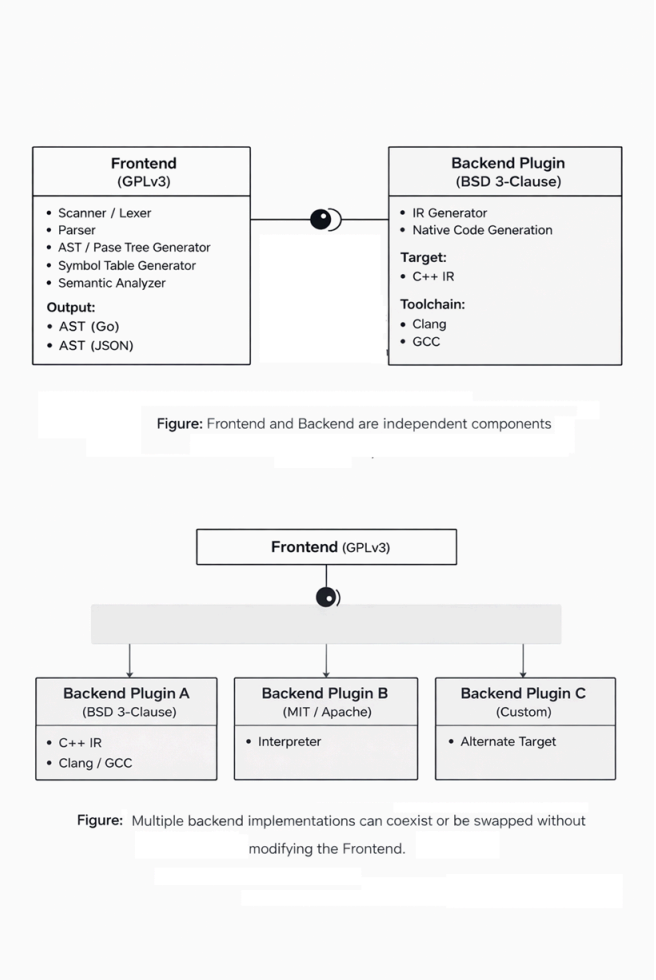


Figure: Frontend and Backend are independent components

Figure: Multiple backend implementations can coexist or be swapped without modifying the Frontend.

FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns**, **license isolation**, and **extensibility through well-defined integration boundaries**.

Compiler and Backend

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Components

- Scanner / Lexer
- Parser
- AST / Parse Tree Generator
- Symbol Table Generator
- Semantic Analyzer

Implementation

- Implemented in **Go**
- Generates AST representations in **Go structures** or **plain JSON**

License

- **GNU General Public License v3 (GPLv3)**

---

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

> **Default Backend shouuld be downloaded/build separately. They are not bundled with Frontend Binary**

Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

Implemented in any language. The frontend emits HIR over an IPC boundary in the declared wire format. Config declares the full protocol, schema, and wire format.

Default Backend

- Backend orchestration implemented in **Go**
- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

3rd Party Backends can have their own licensing terms and implementation choices. Default backend has the following license. **Default backend is not part of complete compiler binary and is separate** should be downloaded and configured using configuration file.

- **BSD 3-Clause License**

Configuration File Structure

Informs the frontend how to generate IR to be consumed by backend. This process is not different for default backend.

```
{
  "protocol":   "folang-plugin/1.0",
  "hir_schema": "folang-hir/1",
  "wire":       "protobuf"
}
```

Licensing Summary

Layer	Responsibility	Implementation	License
Frontend	Parsing and semantic analysis	Go	GPLv3
Backend (default)	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause

---

3. Capability Security Model

FoLang's compiler ships with all language features compiled in but **systems and FFI features are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Feature Tiers

Tier	Features	Default State
`application`	All standard language features, `co.net`, `co.core`, `co.encoding`, `co.crypto`, etc.	✅ Always enabled

Tier	Features	Default State
`system`	Raw pointers, pointer arithmetic, `co.sys.unsafe`, MMIO, heap allocators	🔒 Disabled — requires install-time configuration
`ffi`	`@co.dap.native`, `co.sys.ffi`, extern types, `co.lang.void` pointers, C ABI	🔒 Disabled — requires install-time configuration

Quick Start

Hello World

```
// hello.fol — entry file, no annotation needed
co.out.println("Hello FoLang!")
```

Or with an alias for shorter form:

```
@co.ddap.alias(co.out, as="out")

out.println("Hello FoLang!")
```

Variables

```
// typed
name co.lang.string = "Rao";
age  co.lang.int    = 30;

// inferred from value — := errors if already declared
name := "Rao";
age  := 30;

// define infer and assign if not defined, otherwise assing new value
name ?= "Kumar";
```

`co.lang.string` and `co.lang.int` are Builtin Data types to know more about Builtin Data types, plese refer secion Builtin Data Types

`=`, `:=` and `?=` are built in operators to know more about Built in operators please refer section Builtin Operators

Single Source Application File

FoLang developers can create a complete executable program in one source file. A **single-source application** is an application whose entry file contains the complete program and does not depend on user package source files.

A single-source application file and an application entry file use the same entry-file grammar, context, and restrictions. This section presents the allowed constructs, so a developer can start programming without first reading the complete specification.

Allowed Constructs

The application file may contain:

- built-in directives that are valid for an entry file
- package and library imports
- import aliases declared with `as=`
- file-local aliases for `co.*` paths declared with `@co.ddap.alias`
- type aliases declared with `co.lang.type`
- new types declared with `co.lang.newtype`
- opaque types declared with `co.lang.opaquetype`
- dependent-type aliases and dependent-type usages that do not declare a type-constructor function
- subtype declarations
- supertype declarations
- non-capturing entry-local function-pattern groups

- capturing entry-local `let` function-pattern groups
- variable declarations, initialization, assignment, and mutation
- calls to built-in methods and functions
- calls to imported package and library APIs
- ordinary expressions
- comprehensions
- ternary expressions
- conditions and loops
- containment operations such as `contains`
- iterators such as `each`
- pattern matching and destructuring

```
a co.lang.int = 10;
b co.lang.int = 20;

co.out.println( a + b);
```

co is a reserved word as well as built in top level package

> For more details about reserved words please refer section Reserved Words

Built-in and Imported Names

All `co.*` paths are always available.

A developer may use the complete built-in path:

```
co.out.println("Hello");
co.core.list.of(1, 2, 3);
```

A developer may optionally create a file-local alias:

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")

out.println("Hello");
values := list.of(1, 2, 3);
```

Creating an alias does not hide the complete `co.*` name; both forms remain valid in that file.

Third party packages User packages and libraries are not automatically available. They must be imported using `@co.ddap.import`. When `as=` is present, the imported API is accessed through that alias. When `as=` is omitted, the complete imported package or library path must be used.

```
@co.ddap.import(package="hr.employee", as="emp")
first := emp.EmployeeService.find(1001);

@co.ddap.import(package="finance.payroll")
second := finance.payroll.calculate(request);
```

A developer needs to import the package to use

> `@co.ddap.import` and `@co.ddap.alias` are built in directives to know more about built in directives please refer section Built-in Directives

> for more details on `folang` import directive please check Import details

> `println` is a built in method in `folang` for more details please check Built in Methods `out` is a built in package in `folang` for more details please check Built in Packages

Variable Kinds

`folang` supports different kind of variables for different purposes, even though language provides, developers cannot use at random places. For more information where they are supported refer section Variable Kind support

Simple Variable Declaration

```
someVar co.lang.int;
someString co.lang.string;
```

With Initialization

```
someBool co.lang.bool = co.const.true;
someInt co.lang.int = 42;
```

With Type Inference

```
someVal := "Hello, World!";
someNum := 3.14;   // if not defined, define and initialize; else throws error
someR ?= "Kamesh"; // if not defined, define and initialize; else reassign
```

Pointer Declaration

```
somePtr    co.lang.int->(*);
someDblPtr co.lang.int->(**);
```

Array Declaration

```
someArray       co.lang.int->([5]); // single dimension
someDblArray    co.lang.int->([2,3]); // multi dimension
someJaggedArray co.lang.int->([2][3]); // jagged
someVLArray     co.lang.int->([...]); //varialbe length
someZeroLA      co.lang.int->([0]); //zero length array
someZeroDimA    co.lang.int->([.]); // zerodimension array
```

Array Declaration with Initialization

```
someInitializedArray    co.lang.int->([3])  = [1, 2, 3];
someInitializedArray1   co.lang.int->([])   = [1, 2, 3];
someInitializedDblArray co.lang.int->([,])  = [[1, 2], [3, 4]];
```

Reference Declaration

```
someRef       co.lang.int->(&);    // reference
someLValueRef co.lang.int->(&&);   // LValue reference
someHpRef     co.lang.int->(~);    // heap allocated reference
someAddr      co.lang.int->(@);    // address
someThunk     co.lang.int->(^);    // thunk
someSlice     co.lang.int->([:]);  // slice
```

Range Declaration

```
// Typed range variable declaration
someRange co.lang.int->(..);

// Inferred range declarations
rangeI := 1..10;      // [1, 10]   ExcludeStart=false, ExcludeEnd=false
rangeS := 0<..5;      // (0, 5]    ExcludeStart=true,  ExcludeEnd=false
rangeL := 0..<100;    // [0, 100)  ExcludeStart=false, ExcludeEnd=true
rangeB := 0<..<100;   // (0, 100)  ExcludeStart=true,  ExcludeEnd=true
rangeE := ..100;      // open lower bound  (_, 100]
rangeF := 1..;        // open upper bound  [1, _)
```

Auto and Dynamic Variable Declaration

```
someAutoVar    co.lang.auto    = "Hello"; // type inferred from value; initialization required
someDynamicVar co.lang.dynamic;           // dynamic typing
```

Lazy

```
@co.dap.lazy
x = add(1, 2);  //on doing some thing on x calls add(1,2) till that time add function on right hand side is not
invoked
```

Bind Variables

`$[0-9]*`

Discard / Wildcard Variable

`_`

Comma and Grouping

```
// Comma
x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true;

// Grouping
(x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true);
```

Fat Pointers

```
x co.lang.int->(*, kind="", meta={});

co.lang.int->(*, meta={});

y co.lang.int->(*, meta={len:co.lang.usize, vtab:somepkg.VTable->(*)})

z co.lang.int->(*,kind=region, meta={})
```

```
Pointer
├── base_type: T
├── kind: <FatKind>
│    ├── thin
│    ├── slice
│    ├── relative
│    ├── trait
│    ├── buffer
│    ├── view
│    ├── opaque
│    ├── custom
│    ├── mem
│    ├── nullptr
│    ├── sptr
│    ├── uptr
│    ├── ptrdiff
│    ├── usize
│    ├── ssize
│    └── (region)  ← optional syntactic sugar
└── meta:
     ├── region: heap | stack | global | numa(N) | mmio | constant | …
     ├── len, cap, vtab, bits, endian, …
```

Pointers for address manipulation

```
y co.lang.word->(repr=intptr);
z co.lang.word->(sign=unsigned, repr=uintptr);
p co.lang.word->(repr=ptrdiff);
n co.lang.word->(sign=unsigned, repr=usize);
m co.lang.word->(repr=isize);
o co.lang.void->(repr=nullptr);
```

Relative Pointers

```
z co.lang.int->(*,kind=relative, meta={})
```

Note Other than normal variable declaration rest have restrictions

Conditions, Loops and Iterators

Conditions

```
(boolean truth).do({
}).otherwise(boolean truth).do({
}).otherwise.do({
});
```

Loops

```
(boolean truth).loop({
}).otherwise(boolean truth).loop({
}).otherwise.loop({
});
```

Condition and Loop Mix

```
(boolean truth).do({
}).otherwise(boolean truth).loop({
}).otherwise(boolean truth).do({
}).otherwise.loop({
});
```

Ternary Operator

```
s = (boolean truth).return(some var/value).otherwise.return(some val/var);
s = (boolean truth).return(some var/val).otherwise(boolean truth).return(some var/val).otherwise.return(some
var/val);
```

Looping Arrays / Lists / Maps / Ranges

```
arr co.lang.int->([5]) = [6,7,8,9,10];
arr.each(idx, val).do({
    co.out.print(idx);
    co.out.print(" :: ");
    co.out.println(val);
});

arr.each(_, val).do({
    co.out.println(val);
});
```

Array / List / Map / Range — Contains Element

```
arr co.lang.int->([5]) = [35,57,96,81,31];
k co.lang.int = 31;
arr.contains(k).do({
    co.out.println(k);
}).otherwise.do({
    co.out.println("Not Found");
});
```

Comprehensions *(planned)*

```
k := (1..10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- List(1,2,3)).yield(x * 2)         // List(2, 4, 6)
result := for (x <- Set(1,2,3)).yield(x * 2)           // Set(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2)              // Some(10)
result := for (x <- fetchData()).yield(x.process())    // Future

ages  := {"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age)
```

Pattern Matching

```
x co.lang.int = 10;

x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").default("EQ");
x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").case(_=>"EQ");
x.match(co.pattern.Type).case(co.lang.int => ...).case(co.lang.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG => ...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT => "CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(...);

x.match(co.pattern.Any).case(co.lang.int => ...).case(co.lang.float => ...).case(0 => ...).default( ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 => "First Even Prime").default(...);
```

> **Object vs Instance in FoLang:** Instance is from types of class/structs. Objects are anything — functions, classes, structs, types, etc.

> `_` is a special discard/wildcard variable usable only inside pattern matching, contains, and iterator constructs. Elsewhere `_` must be accompanied by an ASCII letter or number.

> `PostiveEvenMatcher` is custom matcher for more details about creating custom matcher please refer to section Custom Matcher

Type Declarations

```
// Alias
x co.lang.type = co.lang.int;

// New
x co.lang.newtype = co.lang.int;

// Opaque
x co.lang.opaquetype = co.lang.int;  //opaque types act like alias with base type from which they declared and act
like distinct type (new types) for others even when two opaque types derived from same base types

// ADT (tagged union)
y co.lang.type = co.lang.int | co.lang.char;

// Subtype / covariant
test co.lang.subtype = co.lang.int;

// Supertype / contravariant
test co.lang.supertype = co.lang.int;
```

Let Bindings

```
y co.lang.int = let({x = 10}).in({x + 1});
y co.lang.int = let({$ = 10}).in({$ + 1});  // $ refers to the value being defined

x co.lang.int = (x + 1).where(x = 10);
x co.lang.int = ($ + 1).where($ = 10);

offset := 100;

let adjust(0) = offset
let adjust(n) = n + offset
```

> `$` is a special identifier usable in ordinary `let` binding expressions for recursive or self-referential expressions.
>
> Ordinary `let` value-binding expressions remain available in language contexts that permit them, but they are forbidden directly in the application entry file. In the entry file, `let` is reserved exclusively for a named function-pattern group that captures at least one surrounding runtime binding. It cannot introduce an anonymous function, a general closure value, or a curried function.

Function Pattern

```
f(Some(x)) => { x + 1 }
f(None())  => { 0 }

// desugars to:
f(v) =>{
    v.match().case(x: Some(x) => x + 1).case(_: None() => 0);
}
```

Function-pattern groups are permitted in the application entry file as restricted entry-local dispatch helpers. A bare group cannot capture surrounding runtime variables. A `let` function-pattern group must capture at least one already initialized entry-file runtime binding and is the only entry-file construct that permits such capture. Neither form permits ordinary function declarations, anonymous functions, general closure values, currying, partial application, or escape as a function value.

> More about Let and function patterns please refer section Let and Function Patterns

> Single source application file is for testing and getting feel of `folang` Real world applications contain more than single source application file they use concepts of abstraction, encapsulation, inheritance and polymorphism at the core with many other features. Also these applications depend on external libraries, packages to achive clear boundaries with above features. To develop these kind of applications we need following features at minimum.

1. package source files under packages
2. Entry File
3. Libraries and Library surface files
4. imports

Fολang Supports many features to develop enterprise application with intent and FoLang Philosophy — Uniform Object Model are listed below

Complete Feature list:

1. Packages
2. UDT
3. Functions
4. Units
5. Imports
6. Macros
7. Templates
8. Annotations and Decorators
9. Type Classes
10. Types
11. Generics
12. Matchers
13. Lambdas
14. Execution Models and Control Abstractions
15. Extensions
16. Native code
17. Indexers
18. Type Constructors
19. Dynamic Runtime
20. Local/Nested Types and Functions
21. Libraries
22. Operators
23. Forward / Extern Declarations
24. Labels and Named Blocks
25. Reflections
26. Comprehensions

In `folang` User defined data types, function, macros, extensions, templates, typeclasses, type constructors, and units must be in its own file and under a package these files are called package source files. , Lets discuss packages before going to UDTs and Functions

Packages

Package Identity

A subfolder containing `.fol` files **is** a package.

- Dot paths start from subfolders.
- The project root is **not** a package.
- The root folder name never appears in any package dot path.

Examples:

```
/appl/hr/           -> package "hr"
/appl/hr/employee/  -> package "hr.employee"
/appl/auth/         -> package "auth"
```

The project root itself is **not** a package.

Multi-File Packages

Multiple `.fol` files in the same subfolder automatically belong to the same package:

```
hr/employee/
├── Employee.fol      →  hr.employee
├── EmpService.fol    →  hr.employee
└── EmpValidator.fol  →  hr.employee
```

Application Project Layout

```
/appl/
├── app.fol                     ←  entry file — not a package
├── hr/                         package "hr"
│   ├── employee/               package "hr.employee"
│   │   ├── Employee.fol
│   │   └── EmpService.fol
│   └── payroll/                package "hr.payroll"
│       ├── Payroll.fol
│       └── PayrollCalc.fol
├── auth/                       package "auth"
│   └── Auth.fol
└── bindings/                   package "bindings"
    └── CLib.fol
```

Package Aliasing

To alias/change the package name

1. Change folder name(s)
2. The needed subfolder of surface/entryfile folder must contain package.fol file (Need revisit **planned**)

```
  requiredname co.lang.package;
```

> More about what a packages please refer section Packages in detail.

UDT (User defined Data types)

`folang` provides following constructs to create User Defined Data types, as mentioned UDTs must be in their own source file under package.

1. cstructs
2. structs
3. unionns
4. enums
5. classes
6. Modules
7. interface
8. signature

> For more information about UDTs please refer section Built In Kinds

---

Struct Declaration

```
myStruct co.lang.struct={
    field1 co.lang.int;
    field2 co.lang.string;
    field3 co.lang.bool;
}
```

> More about structs please refer section `Structs in detail`

C-Struct Declaration

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
Point co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}

Rect co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

Enum Declaration

```
myEnum co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

Union Declaration

```
myUnion co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Class Declaration

```
Employee co.lang.class ={
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating — internally redirecting the call to module function
}

// $1, $2, $3 ... are previous results captured as bind variables
Emp co.lang.class={
    dosomething(a co.lang.int, b co.lang.int)->
(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

> More about classes please refer section `Classes in detail`

Interface vs Signature

```
// Employee is an ordinary package-level declaration.
MEmployee co.lang.signature = {
    ...
}
```

> For more about signatures please refer section `signatures in detail`

```
IEmployee co.lang.interface = {
  ...
}
```

> For more about interfaces please refer section `interfaces in detail`

Module Declaration

```
// Employee.fol — ordinary package-level type
Employee co.lang.struct = {
  ...
}

// EmployeeModule.signature.fol
EmployeeModule co.lang.signature = {
  ...
}

// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
EmployeeModImpl co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

   ...
}
```

> More about modules please refer section `Modules in detail`

Units

Unit Declaration 🟩

A `unit` is a named, non-instantiable container for functions.

Like other named primary declarations, a unit may use an explicit name or `_` for filename-derived naming. For example, `_ co.lang.unit` in `Math.fol` declares `Math`, while an explicit name such as `Math co.lang.unit` is authoritative regardless of the filename.

Its purpose is structural: it prevents functions from flowing freely at package-file scope and preserves FoLang's rule that every ordinary package source file has one primary top-level declaration. A unit does **not** require its functions to form a domain model, capability, service, or other semantic abstraction.

Functions within a unit may be strongly related by purpose—for example, mathematical, parsing, encoding, or validation functions—or only loosely related. Semantic cohesion is encouraged as a source-design practice but is not enforced by the language.

A unit has two forms:

1. **Standalone unit** — its name does not match a struct in the same package. It acts as an ordinary named container for receiverless functions.
2. **Struct companion unit** — its name matches exactly one `co.lang.struct` in the same package. It provides behaviour associated with that struct while the struct declaration itself remains pure data.

Only `co.lang.struct` can have a companion unit. A same-name unit is not allowed for `co.lang.class`, `co.lang.cstruct`, modules, enums, unions, interfaces, signatures, or other declaration kinds. Classes already contain their own methods and lifecycle behaviour; `cstruct` remains a restricted C-compatible data representation.

```
Text co.lang.unit={

}
```

> for more about units please refer section `units in detail`

Matchers

Custom Matcher

```
@co.dap.matcher
Matcher(T) = {
    matchCase(value T, pattern co.lang.untyped) -> (co.lang.int, co.lang.MatchBindings);
    // int return: 0 = no match, >0 = match
}

PositiveEvenMatcher co.lang.Matcher->(for=Matcher, type=co.lang.int) = {
    matchCase(value co.lang.int, pat co.lang.untyped)->(co.lang.int, co.lang.MatchBindings) = {
        // user logic
    }
}
```

Comprehensions *(planned)*

```
k := (1..10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- List(1,2,3)).yield(x * 2)         // List(2, 4, 6)
result := for (x <- Set(1,2,3)).yield(x * 2)           // Set(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2)              // Some(10)
result := for (x <- fetchData()).yield(x.process())    // Future

ages  := {"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age)
```

Extensions

```
stringextension co.lang.unit={

    @co.dap.extension(fortype=co.lang.string, what=extends)
    upperCase()->(string)={
        return this.upper()
    }

    @co.dap.extension(fortype=[co.lang.string], what=overrides)
    equals(str string)->(bool)={
        this.return this == str
    }
}
```

Extensions must be **explicitly activated** — they are block-scoped:

```
@co.ddap.use(from="stringextension",extensions=[equals, upperCase])
k.upperCase();  // ✅ explicitly activated
```

Reflections

```
@co.dap.reflection(enable=True, package="co.meta")

x co.lang.int = 10;
x.reflect().getType()  → co.lang.int
x.reflect().getValue() → 10
x.reflect().getKind()  → value
```

Type Classes

Monads, Applicatives, Functors, Monoids and Transformers

> `@co.dap.typeclass(kind=...)` is the single annotation for all typeclass definitions. `kind` specifies the algebraic structure — `Functor`, `Applicative`, `Monad`, `Monoid`, `Transformer`, or any user-defined kind. Instances of any typeclass always use `co.lang.instance`.

Functor

```
@co.dap.typeclass(kind=Functor)
Functor(F) = {
    map(value F(A), f (A)->B) -> (F(B));
}

ListFunctor co.lang.instance->(for=Functor, type=List) = {
    map(value List(A), f (A)->B)->(List(B)) = {
        result = List(B){}
        value.each(_, item).do({ result.append(f(item)) });
        this.return result
    }
}
```

Applicative

```
@co.dap.typeclass(kind=Applicative)
Applicative(F) = {
    pure(x A) -> (F(A));
    apply(fab F(A->B), fa F(A)) -> (F(B));
}

OptionApplicative co.lang.instance->(for=Applicative, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    apply(fab Option(A->B), fa Option(A))->(Option(B)) = {
        this.return (fab, fa)
            .match
            .case((Some(f), Some(x)) => Some(f(x)))
            .default(None());
    }
}
```

Monad

```
@co.dap.typeclass(kind=Monad)
Monad(F) = {
    pure(x A) -> (F(A));
    flatMap(fa F(A), f (A)->F(B)) -> (F(B));
}

OptionMonad co.lang.instance->(for=Monad, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    flatMap(fa Option(A), f (A)->Option(B))->(Option(B)) = {
        this.return fa.match().case(Some(x) => f(x)).default(None);
    }
}
```

Monoid

```
@co.dap.typeclass(kind=Monoid)
Monoid(T) = {
    empty() -> (T);
    combine(a T, b T) -> (T);
}

IntMonoid co.lang.instance->(for=Monoid, type=co.lang.int) = {
    empty()->(co.lang.int) = { this.return 0; }
    combine(a co.lang.int, b co.lang.int)->(co.lang.int) = { this.return a + b; }
}
```

Transformer

```
@co.dap.typeclass(kind=Transformer)
Transformer(F(_), G(_)) = {
    map(value F(A), f (A)->B) -> (G(B));
}

ListToSetTransformer co.lang.instance->(for=Transformer, types=[List, Set]) = {
    map(value List(A), f (A)->B)->(Set(B)) = {
        result = Set(B){}
        value.each(_, item).do({ result.insert(f(item)) });
        this.return result;
    }
}
```

Reflection

```
@co.dap.reflection(enable=True, package="co.meta")

x co.lang.int = 10;
x.reflect().getType()  → co.lang.int
x.reflect().getValue() → 10
x.reflect().getKind()  → value
```

Labels and Named Blocks

```


// Label
outer:{
    // statements
}

//Blocks
 {
    // statements
 }

// Named Block
labelBlock co.lang.block={

}

labelBlock.expand();
```

---

imports

FoLang supports three import forms. User packages and libraries must be imported before use. When an import declares `as=`, symbols are accessed through that alias. When `as=` is omitted, the complete imported package or library path is used.

1. Normal Package Import

Use this for ordinary project packages.

```
@co.ddap.import(package="hr.employee", as="emp")

e := emp.getEmployee(1);
co.out.println(e.name);
```

Resolution:

```
package="hr.employee" -> /appl/hr/employee/
```

2. Source Library Import

Use this for same-owner workspace libraries whose source is available.

```
@co.ddap.import(package="com.abc.ffi", src-library=true, expect="ffi", as="ffilib")
```

Resolution:

```
package="com.abc.ffi", src-library=true -> /appl/com/abc/ffi.fol
```

The resolved file must be a library surface file:

```
@co.dap.library(type="ffi")
ffilib co.lang.library={

}
```

Meaning:

- `package` gives the logical library path
- `src-library=true` means the leaf resolves to a single `.fol` surface file, not a folder
- `expect` is an import-site assertion; the compiler checks it against the actual library type
- only the projected surface API is visible; internal sources are not importable through normal package imports

3. Packaged Library Import

Use this for third-party or prebuilt libraries.

```
@co.ddap.import(library="hrlib", as="hr")
@co.ddap.import(library="paylib", as="pay")
```

Resolution:

```
library="hrlib" -> libs/hrlib.folenc, else libs/hrlib.folib
```

Only the packaged library's projected surface API is visible to the consumer.

Import Directive Fields

Field	Required	Default	Meaning
`package` or `library`	one required	—	logical package path or packaged library name
`src-library`	❌	`false`	when `true`, `package=` resolves to a source library surface file
`expect`	❌	inferred from library surface	expected library kind such as `ffi`, `system`, `advanced`, `dynamicvmrt`, or `application`
`as`	❌	none — full dot path required when omitted	local alias; valid FoLang identifier
`realm`	❌	`app`	isolation domain

Field	Required	Default	Meaning
`parent-realm`	❌	—	realm shadowing relationship

Notes:

- `as` is optional — when omitted, no short alias is created and the full imported package path must be used to access symbols
- dots are not allowed in `as`
- `expect` is validation, not the source of truth
- the source of truth is `@co.dap.library(type="...")` on the resolved surface file

Examples:

```
@co.ddap.import(package="hr.employee", as="emp")
emp.Employee

@co.ddap.import(package="hr.employee")
hr.employee.Employee
```

Valid `as` Values

```
as="hr"       ✅
as="v1_hr"    ✅
as="v1.hr"    ❌
as="123hr"    ❌
```

Realms

Realms provide import isolation, coexistence of versions, and controlled shadowing.

Realm declarations are always syntactically valid — you can always write `realm=` on any import. However realm isolation is **active only when the library marked with `dynamicvmrt`**. Without it, realm declarations are not valid unless its library is `dynamicvmrt` and throw compiler error.

Default realm:

```
app
```

Hierarchy example:

```
core
  └── app
        ├── app1
        │     └── app3
        └── app2
```

Rules:

- `realm` defaults to `app`
- if `parent-realm` is omitted, parent defaults to `app`
- `parent-realm` is meaningful only when `realm` is explicitly provided and is not `app`
- libraries are always static
- `@co.ddap.dynamicruntime` is allowed only on the application entry file

Version Coexistence

```
@co.ddap.import(package="hr", realm="app", as="hr")
@co.ddap.import(package="v1.hr", realm="x", as="v1_hr")
```

Shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

When the alias is the same and the child realm points to the parent realm, the child shadows the parent.

Import Binding Rules

Invalid

Same alias, same realm, different package:

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="b", realm="app", as="hr")  // error
```

Valid aggregation

```
@co.ddap.import(package="folderx.some", realm="app", as="hr")
@co.ddap.import(package="folderx.some", realm="app", as="hr")
```

Valid coexistence

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", as="v1_hr")
```

Valid shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

Cycles

Compiler error if any cycle exists through:

- package imports
- realm parent relationships

Examples:

- `packageA` imports `packageB`, and `packageB` imports `packageA`
- `realm="x", parent-realm="y"` and another import uses `realm="y", parent-realm="x"`

Symbol Resolution

Resolution Order

For symbols without a prefix:

1. current package symbol table
2. parent package chain upward
3. error if not found

For symbols with an alias prefix:

1. current package imported-context map
2. parent package chain imported-context maps
3. error if not found

For imports declared **without** `as`:

1. use the full imported package path directly
2. resolve it through imported-context lookup using that full path
3. error if not found

Example Context Graph

```
pp1 (grandparent package)
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

p1 (parent package)
    ├── Parent -> &pp1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

c1 (current package)
    ├── Parent -> &p1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs
```

Resolution Examples

```
lookup "OwnType"
    -> current symbol table

lookup "ParentType"
    -> parent chain

lookup "hr.Employee"
    -> imported-context lookup for alias "hr"

lookup "hr.employee.Employee"
    -> imported-context lookup for full imported package path when no alias exists

lookup "unknown.Type"
    -> imported-context lookup fails -> compiler error
```

Short Summary

- folders define packages
- root is never a package
- each ordinary package source file has exactly one primary top-level declaration
- free functions in package source files must be enclosed in a `co.lang.unit`
- the application entry file is an executable, non-package context with its own restricted declaration rules
- entry-local type aliases, newtypes, opaque types, dependent-type aliases/usages, subtypes, supertypes, bare function-pattern groups, and capturing `let` function-pattern groups are allowed
- ordinary `let` value bindings, ordinary functions, anonymous functions, general closures, classes, structs, enums, cstructs, type constructors, generics, macros, templates, units, and reusable behavioral declarations are forbidden in the entry file
- `co.*` is always available and never imported
- `@co.ddap.alias` optionally shortens a `co.*` path; otherwise the complete path is used
- `@co.ddap.import(package="...")` imports normal packages
- `@co.ddap.import(package="...", src-library=true, ...)` imports same-owner source libraries
- `@co.ddap.import(library="...")` imports packaged external libraries
- `expect="..."` is an import-site assertion, not the source of truth
- `@co.dap.library(type="...")` is the source of truth for library kind
- every library surface exports only boundary `struct`/`cstruct` contracts and public function signatures
- surface function bodies are restricted boundary adapters and are hidden from consumer symbol tables
- application-family boundary structs cross by automatic deep snapshot
- system and FFI boundary cstructs cross by ABI value
- internal packages never depend on surface types; the surface converts between public and internal representations

---

Let and Function Patterns

Bare Function-Pattern Group

A bare function-pattern group does not capture surrounding runtime bindings:

```
classify(0) => { "zero" }
classify(n).where(n > 0) => { "positive" }
classify(_) => { "negative" }
```

A bare group may use:

- its parameters and names introduced by its patterns
- its own name for recursion
- entry-local type names
- `co.*` APIs
- imported package and library APIs
- compile-time symbols that do not require runtime capture

It may not reference an entry-file runtime variable from the surrounding context:

```
offset := 100;

adjust(n) => { n + offset }
// compiler error: bare function-pattern groups cannot capture `offset`
```

Capturing `let` Function-Pattern Group

The `let` form exists only to declare an entry-local function-pattern group that captures one or more surrounding entry-file runtime bindings:

```
offset := 100;

let adjust(0) = offset
let adjust(n) = n + offset

result := adjust(10);
```

The captured names must resolve to surrounding runtime bindings that are already declared and definitely initialized before the first clause of the group. Built-in names, imported declarations, type names, parameters, and the function's own recursive name are not captures.

A `let` function-pattern group must capture at least one surrounding runtime binding. When no capture is required, the bare form must be used:

```
let fib(0) = 1
let fib(1) = 1
let fib(n) = fib(n - 1) + fib(n - 2)
// compiler error: this group captures nothing; remove `let`

fib(0) => { 1 }
fib(1) => { 1 }
fib(n) => { fib(n - 1) + fib(n - 2) }
```

The `let` marker is therefore not an optional spelling of a bare function-pattern group. It explicitly requests restricted lexical capture.

Similarities

Bare and capturing `let` function-pattern groups both:

- group compatible clauses by function name and arity
- dispatch by parameter patterns and optional `.where(...)` guards
- evaluate clauses in normal pattern-selection order
- may call themselves recursively
- must maintain compatible parameter and result types across all clauses

- follow the normal pattern ordering, reachability, overlap, and exhaustiveness rules
- are private to the entry file
- cannot be imported, exported, or annotated with package visibility
- cannot be converted to, assigned as, passed as, or returned as function values
- cannot be partially applied or curried

Differences

Form	Surrounding runtime capture	Intended use
`name(pattern) => body`	No	Entry-local pattern dispatch that depends only on its arguments, recursion, built-ins, imports, and compile-time names
`let name(pattern) = body`	Yes, at least one capture required	Entry-local pattern dispatch that also depends on existing entry-file runtime bindings

Clauses of the same name and arity must all use the same form. Mixing bare and `let` clauses in one function-pattern group is a compiler error.

A capturing `let` function-pattern group is still not a general closure facility. It is named, entry-local, non-first-class, and non-escaping. The compiler may internally retain a capture environment, but FoLang does not expose that environment as a closure object.

`let` cannot be used directly in the entry file for value bindings, anonymous functions, general closure expressions, or curried functions:

```
let base = 10;                  // compiler error: entry-file let value binding
let result = base + 1;          // compiler error: entry-file let value binding
let add = (a, b) => a + b;      // compiler error: anonymous function
let counter = closure { ... };  // compiler error: general closure expression
let add = a => b => a + b;      // compiler error: curried function
```

The compiler may lower either function-pattern form to a private entry helper, but neither source construct is a general-purpose function declaration.

Package in detail

Package Identity

A subfolder containing `.fol` files **is** a package.

- Dot paths start from subfolders.
- The project root is **not** a package.
- The root folder name never appears in any package dot path.

Examples:

```
/appl/hr/           -> package "hr"
/appl/hr/employee/  -> package "hr.employee"
/appl/auth/         -> package "auth"
```

The project root itself is **not** a package.

It may contain:

- the application entry file which is same as single source application file
- the packaged library surface file when the project itself is a library
- subfolders that define packages or same-owner source libraries

Multi-File Packages

Multiple `.fol` files in the same subfolder automatically belong to the same package:

```
hr/employee/
├── Employee.fol      →  hr.employee
├── EmpService.fol    →  hr.employee
└── EmpValidator.fol  →  hr.employee
```

Application Project Layout

```
/appl/
├── app.fol                     ←  entry file — not a package
├── hr/                         package "hr"
│   ├── employee/               package "hr.employee"
│   │   ├── Employee.fol
│   │   └── EmpService.fol
│   └── payroll/                package "hr.payroll"
│       ├── Payroll.fol
│       └── PayrollCalc.fol
├── auth/                       package "auth"
│   └── Auth.fol
└── bindings/                   package "bindings"
    └── CLib.fol
```

Package Access Rules

Four access levels control visibility across package boundaries:

Annotation	Same Package	Parent	Sub-Package	Unrelated	Entry / Surface
`@co.dap.public`	✅	✅	✅	✅	✅
`@co.dap.package`	✅	✅	✅	❌	❌
`@co.dap.protected`	✅	❌	✅	❌	❌
`@co.dap.private`	✅	❌	❌	❌	❌

Meaning:

- `@co.dap.public` -> visible everywhere
- `@co.dap.package` -> visible across the package family
- `@co.dap.protected` -> visible downward into subpackages only
- `@co.dap.private` -> visible only inside the declaring package

Example:

```
// hr/EmployeeAccess.fol — package "hr"

@co.dap.public
EmployeeAccess co.lang.unit = {

    @co.dap.public
    getEmployee(id co.lang.int)->(Employee) = { ... }     // anyone can call

    @co.dap.package
    validateId(id co.lang.int)->(co.lang.bool) = { ... }  // hr package family

    @co.dap.protected
    baseQuery()->(co.lang.string) = { ... }               // visible to subpackages

    @co.dap.private
    normalizeId(id co.lang.int)->(co.lang.int) = { ... }  // private declaration
}
```

`co.*` Paths and Aliases

`co.*` Is Always Available

All `co.*` paths are part of the language and are always in scope. They are never imported through `@co.ddap.import`.

```
co.out.println("hello")
co.in.readln()
x co.lang.int = 42;
```

Being **in scope** does not automatically mean **permitted in every context**.

A package rule, library kind, or other compiler restriction may still forbid use of a particular `co.*` facility. In such cases the name is still known to the compiler, but its use is rejected at compile time.

`@co.ddap.alias`

Use aliases only to shorten `co.*` paths.

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")
@co.ddap.alias(co.encoding, as="enc")

out.println("hello")
list.of(1, 2, 3)
enc.json.serialize(data)

// full form still works alongside
co.out.println("world")
```

Rules:

- target must be a `co.*` path
- alias must be a valid identifier
- alias scope is file-local
- aliasing user packages is not allowed; use `as=` in `@co.ddap.import`
- duplicate aliases in the same file are compiler errors
- when no alias is declared, the complete `co.*` path is used
- declaring an alias does not disable or hide the complete `co.*` path

---

Package Source Files

A `.fol` file located inside a package folder contains **exactly one primary top-level declaration**. This gives every package source file a clear structural identity and prevents unrelated declarations or loose functions from being mixed at file scope.

The primary declaration may be a:

- class
- struct
- cstruct
- enum
- union / ADT
- interface
- signature
- module
- type classes/instance
- type constructors
- patterns or objects
- macro
- template
- generics

- annotations
- decorators
- type, type alias, newtype, or opaque type
- unit

File-level import directives, aliases, and annotations may appear before the primary declaration. They do not count as additional primary declarations.

Forbidden directly at package-file scope:

- free-flowing functions
- variables or mutable state
- executable statements
- explicit package declarations
- project metadata
- library metadata
- multiple unrelated primary declarations

The application entry file and library surface files are special source forms and follow their own rules. The single-primary-declaration rule in this section applies to ordinary files inside package folders.

Primary Declaration Names and Filename Inference

The name of a primary declaration may be written explicitly or inferred from the source filename.

When `_` appears in the primary declaration-name position, the compiler derives the declaration name from the filename:

```
// Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

This declares `Employee co.lang.struct`.

In this position, `_` does not declare an anonymous or discarded declaration. It means **use the filename-derived declaration name**.

An explicit declaration name is authoritative and overrides filename inference. The explicit name is not required to match the filename:

```
// employee_model.fol
Employee co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

This still declares `Employee`; `employee_model.fol` is only the source filename.

Name-selection precedence is therefore:

```
explicit declaration name  -> use the explicit name
_                          -> derive the name from the filename
```

Filename derivation rules:

- for `Name.fol`, the inferred declaration name is `Name`
- for a kind-qualified filename such as `Name.unit.fol`, the compiler removes both `.fol` and the trailing kind suffix when that suffix matches the declaration kind
- the inferred result must be a valid FoLang identifier
- a kind-qualified filename whose suffix conflicts with the declaration kind is a compiler error
- filename inference supplies only the declaration name; generic parameters and all other declaration details remain explicit

Examples:

```
// Status.enum.fol
_ co.lang.enum = {
    active;
    inactive;
}
// inferred name: Status
```

```
// Box.struct.fol
_(T) co.lang.struct = {
    value T;
}
// inferred name: Box; T remains explicit
```

```
// Employee.struct.fol
_ co.lang.class = {
    ...
}
// compiler error: filename kind `struct` conflicts with declaration kind `class`
```

Filename inference may be used by named primary declarations such as classes, structs, cstructs, enums, unions, interfaces, signatures, modules, instances, objects, units, and named type declarations.

Example containing a data declaration:

```
// hr/employee/Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

When a file needs only ordinary free functions, those functions must be enclosed in a `co.lang.unit`:

```
// hr/employee/EmpService.fol
_ co.lang.unit = {

    getEmployee(id co.lang.int)->(Employee) = {
        this.return Employee{ id: 1, name: "Rao" };
    }
}
```

Application Entry File

The application entry file is a **special executable source form** located at the application root. It is not a package, does not create an importable namespace, and is not subject to the ordinary package-file rule requiring exactly one primary declaration.

The compiler creates a dedicated **entry-file context** for it:

```
ApplicationEntryContext
├── file directives, imports, and aliases
├── entry-local non-structural type declarations
├── non-capturing function-pattern groups
├── capturing `let` function-pattern groups
└── executable statements and expressions
```

Everything declared directly in this context is private to the entry file. Entry-local symbols cannot be imported, exported, or referenced by package or library source files.

Allowed Entry-File Constructs

The application entry file uses exactly the same grammar and allowed-construct rules described in Single Source Application File. The earlier section provides the developer-facing overview; this section defines the entry file's formal context, privacy, and dependency direction.

Entry-Local Function Patterns

Function-pattern groups are allowed as a special entry-file construct even though ordinary function declarations are forbidden. FoLang provides two entry-file forms with the same pattern-dispatch model but different capture semantics.

Entry-File Dependency Direction

The entry file may depend on packages and libraries, but packages and libraries may never depend on the entry context:

```
application entry file
    ↓ uses
packages and libraries
```

This allows the entry file to coordinate application startup while preserving package and library independence.

Libraries

Library Project Layout

```
/hrlib/
├── hrlib.fol                   ←  library surface — @co.dap.library
├── emp/                        package "emp" — internal, invisible to consumer
│   ├── Employee.fol
│   └── EmpService.fol
└── auth/                       package "auth" — internal, invisible to consumer
    ├── Auth.fol
    └── AuthService.fol
```

Consumer only sees what `hrlib.fol` declares. All subfolders are internal.

Library Surface file

FoLang uses surface files in two situations:

1. **Packaged library project surfaces**
2. **Application-workspace source library surfaces**

A surface file is a special source form annotated with `@co.dap.library`. It defines one library identity, its public boundary data contracts, and the boundary-adapter functions through which consumers call the library.

```
app.fol   -> application entry
hrlib.fol -> packaged library surface
ffi.fol   -> source library surface when imported with src-library=true
```

A library surface is not an ordinary package file. It may contain multiple boundary data declarations and public functions inside one `co.lang.library` declaration.

Packaged Library Project Surface

A packaged library project has no application entry file. It has exactly one surface file at the project root.

```
/hrlib/
├── hrlib.fol                   <- library surface
├── emp/                        <- internal package
│   ├── Employee.fol
│   └── EmployeeService.fol
└── auth/                       <- internal package
    └── AuthService.fol
```

Rules:

- exactly one `@co.dap.library` surface file exists per packaged library project
- the surface file is located at the project root
- it is compiled into the packaged library artifact, such as `.folib` or `.folenc`
- consumers import it with `@co.ddap.import(library="...")`
- internal package folders are compiled into the library but are not directly visible to consumers

Application-Workspace Source Library Surface

A source library may live inside an application workspace.

```
@co.ddap.import(
    package="com.abc.ffi",
    src-library=true,
    expect="ffi",
    as="ffilib"
)
```

The import resolves to one surface file:

```
/appl/com/abc/ffi.fol
```

Rules:

- a source-library surface file must be below the application root, never at the application root
- multiple source libraries may exist in one application workspace
- the surface is imported through `package="..."` with `src-library=true`
- only the projected surface API is importable
- internal packages remain hidden even though their source files are physically available
- once a source tree is treated as a library, its subpackages cannot be imported as ordinary packages by consumers
- packaged libraries and source libraries use exactly the same public-surface and API-projection rules

Unified Surface Model

Every library kind uses the same conceptual surface shape:

```
library surface
├── boundary data contracts
└── public boundary-adapter functions
```

Library kind changes the permitted boundary representation and transfer semantics, not the conceptual form of the API.

Library kind	Boundary data	Transfer semantics
`application`	`co.lang.struct`	automatic deep snapshot
`dynamicvmrt`	`co.lang.struct`	automatic deep snapshot
`advanced`	`co.lang.struct`	automatic deep snapshot
`system`	`co.lang.cstruct`	system ABI value
`ffi`	`co.lang.cstruct`	C ABI value

`co.lang.struct` and `co.lang.cstruct` solve different problems:

```
struct  -> FoLang semantic data contract
cstruct -> physical ABI-compatible value contract
```

Value transfer must not be confused with C-compatible layout. Application-family libraries therefore use expressive `struct` contracts transferred as deep snapshots, while system and FFI libraries use restricted `cstruct` contracts.

Allowed Surface Declarations

A library surface may contain only:

- file- or library-level imports needed by its adapter implementations
- `co.lang.struct` boundary declarations for `application`, `dynamicvmrt`, and `advanced`
- `co.lang.cstruct` boundary declarations for `system` and `ffi`
- public free-function API declarations with boundary-adapter definitions

The following declaration kinds are forbidden directly in every library surface:

- classes
- modules
- interfaces
- signatures

- units and companion units
- associated functions
- operator functions
- enums, unions, and other ADTs
- type aliases, newtypes, and opaque types
- objects, instances, type classes, and dependent types
- macros, templates, annotations, and decorators
- global variables, pointers, references, addresses, or mutable surface state

Surface `struct` and `cstruct` declarations are data contracts only. They cannot have companion units, associated functions, operators, methods, or other behavior on the surface.

Declarations directly inside the `co.lang.library` body are exported by default. Imports and implementation details are never exported.

Public Signature Type Closure

Every public surface function signature must be closed over the library's exported boundary type set.

For `application`, `dynamicvmrt`, and `advanced` surfaces, parameters and results may use only:

- approved built-in types
- `co.lang.struct` types declared in the same library surface

For `system` and `ffi` surfaces, parameters and results may use only:

- ABI-safe built-in types
- `co.lang.cstruct` types declared in the same library surface

The same closure rule applies recursively to fields of surface boundary types:

- a surface `struct` field may use an approved built-in type or another surface-declared `struct`
- a surface `cstruct` field may use an ABI-safe built-in type or another surface-declared `cstruct`
- an internal package type may never appear in a public function signature or surface boundary-type field
- pointers, references, and addresses may never cross any public library surface

A built-in type is not automatically surface-safe merely because it belongs to `co.lang`. An approved surface built-in must be concrete, fully resolved, and transferable under the library kind's boundary semantics.

The following categories are forbidden in public surface fields and signatures:

- inference-only types such as `co.lang.auto` and `co.lang.infer`
- dynamically typed or unconstrained carriers such as `co.lang.dynamic`, `co.lang.any`, `co.lang.typed`, and `co.lang.untyped`
- function, closure, delegate, loader, realm, AST, reflection, or runtime implementation values
- pointer, reference, address, thunk, and implementation-handle types
- any type whose reachable representation contains a forbidden type

For application-family surfaces, managed built-ins such as `co.lang.string` are permitted when the compiler defines deep-snapshot reconstruction for them. For system and FFI surfaces, only built-ins with a defined ABI representation are permitted; for example, `co.lang.string` is not directly cstruct-compatible.

Valid:

```
Employee co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address Address;
}

Address co.lang.struct = {
    city co.lang.string;
}
```

Invalid:

```
getEmployee(id co.lang.int)->(emp.internal.Employee);
// compiler error: an internal type escapes through the public signature
```

Boundary-Adapter Functions

A public surface function may contain a definition, but its definition is restricted to boundary adaptation.

A boundary-adapter function may:

- read its input parameters and their fields
- declare temporary local values used only for conversion
- construct internal input values
- perform deterministic field-to-field or representation conversion
- invoke exactly one internal implementation operation
- receive that operation's result
- construct and return an allowed surface `struct`, surface `cstruct`, or built-in value
- use a direct delegation expression when no conversion is required

A boundary-adapter function may not:

- implement business rules or domain calculations
- perform business validation or authorization decisions
- orchestrate multiple implementation operations
- call another surface function
- perform persistence, networking, file I/O, retries, caching, or transactions
- start concurrency, scheduling, asynchronous work, processes, or continuations
- contain loops, recursion, workflow branching, or externally observable state mutation
- expose an internal value directly without converting it to an allowed boundary type

The compiler enforces the restricted adapter statement set. The restriction is structural: arbitrary executable logic is not accepted merely because it is placed in a surface file.

A direct delegate is the simplest adapter form:

```
health()->(co.lang.bool)
    => health.internal.Service.health();
```

A converting adapter may map between the public contract and an internal model.

Application Surface Example

```
// hrlib.fol
@co.dap.library(type="application")
hrlib co.lang.library = {

    @co.ddap.import(package="emp", as="emp")

    Employee co.lang.struct = {
        name co.lang.string;
        id   co.lang.int;
    }

    getEmployee(empId co.lang.int)->(Employee) = {
        internalEmployee := emp.EmployeeService.getEmployee(empId);

        this.return Employee{
            name: internalEmployee.name,
            id: internalEmployee.id
        };
    }
}
```

The surface owns conversion from the internal `emp.Employee` representation to the public `hrlib.Employee` contract. The `emp` package owns validation, business rules, persistence, and workflow.

The consumer sees the equivalent API contract:

```
Employee co.lang.struct = {
    name co.lang.string;
    id   co.lang.int;
}

getEmployee(empId co.lang.int)->(Employee);
```

The consumer does not see the body of `getEmployee` or the `emp` package.

System and FFI Surface Example

```
// driver.fol
@co.dap.library(type="system")
driver co.lang.library = {

    @co.ddap.import(package="driver.internal", as="impl")

    Point co.lang.cstruct = {
        x co.lang.int32;
        y co.lang.int32;
    }

    getOrigin()->(Point) = {
        internalPoint := impl.DriverService.getOrigin();

        this.return Point{
            x: internalPoint.x,
            y: internalPoint.y
        };
    }
}
```

Pointers, addresses, native bindings, and hardware state belong in `driver.internal`. They are forbidden in the public surface and cannot appear in the exported signature.

Boundary Transfer Semantics

For `application`, `dynamicvmrt`, and `advanced` libraries:

```
consumer boundary value
    ↓ automatic deep snapshot of the complete reachable value graph
library-local built-in or surface struct
    ↓ surface conversion
internal implementation value
    ↓ surface conversion
library-local built-in or surface struct
    ↓ automatic deep snapshot
consumer-local boundary value
```

The snapshot rule applies to both surface structs and approved built-in arguments/results. The library never receives the consumer's live object identity, and the consumer never receives a live internal-library object.

For `system` libraries, `cstruct` values cross according to the selected backend/platform system ABI.

For `ffi` libraries, `cstruct` values cross according to the declared C ABI, including its layout, alignment, and calling-convention requirements.

Surface-to-Internal Dependency Direction

The source-level dependency is one-way:

```
library surface
    ↓ imports and invokes
internal packages
```

Internal packages:

- do not import the library surface
- do not use surface-declared `struct` or `cstruct` types
- define their own implementation and domain types
- return internal values to the surface adapter
- contain all business logic, validation, workflow, I/O, and state management

This prevents a surface/internal compilation cycle and keeps the public contract independent from the internal domain model.

An internal implementation symbol may be visible to its own library surface without becoming part of the consumer API. Library encapsulation takes precedence over ordinary package visibility: consumers cannot resolve internal package symbols even when those symbols are callable from the surface during library compilation.

Consumer API Projection

The compiler does not expose the complete surface compilation context to a consumer. It creates a projected imported symbol table.

The projected API contains:

- the library identity and kind
- complete public surface `struct` or `cstruct` definitions, including field names and permitted field types
- public function names
- public function parameter names and types
- public function result types
- calling-convention, effect, error, and linkage metadata where applicable

The projected API does not contain:

- function bodies or implementation AST/HIR
- imports used by the surface
- local conversion variables
- delegate or implementation targets
- internal package paths or symbols
- private compiler-generated helpers
- business classes, modules, units, or internal data types

Therefore, a function definition may exist in the surface source and compiled artifact while the consumer's symbol table contains only its signature.

The backend may retain implementation bodies for code generation, linking, or whole-program optimization. Backend possession of implementation code does not make that code semantically visible to the consumer.

Source libraries follow the same rule. Availability of source code does not weaken the projected API boundary.

Library Compilation Order

A library is processed in four logical stages:

1. parse the surface header, boundary data declarations, and public function signatures
2. compile internal packages without depending on surface types
3. type-check and link surface boundary-adapter bodies against internal package symbols
4. emit the compiled implementation and the projected consumer API

This order permits the surface to call internal packages while preventing internal packages from depending on the surface.

Library Kinds

Library kinds are declared on the surface file:

```
@co.dap.library(type="ffi")
@co.dap.library(type="system")
@co.dap.library(type="dynamicvmrt")
@co.dap.library(type="advanced")
@co.dap.library(type="application")
```

Shared Meaning

- `application` -> ordinary safe library implementation
- `dynamicvmrt` -> application features plus full `co.meta` dynamic-runtime support
- `advanced` -> macro, template, concurrency, transformation, and runtime-machinery implementation
- `system` -> unsafe low-level runtime, pointer, process, native, and platform-control implementation
- `ffi` -> foreign-interface implementation

Library kind controls internal capabilities. All kinds still expose only boundary data contracts and public boundary-adapter functions.

`application` Libraries

Internally allowed:

- ordinary safe language features
- classes, modules, interfaces, signatures, units, and ordinary structs
- normal application-level packages and libraries

Internally forbidden:

- pointers, references, and addresses
- native functions annotated with `@co.dap.native`
- macros and templates
- low-level concurrency machinery
- advanced transformation or dynamic-runtime machinery

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`dynamicvmrt` Libraries

Internally allowed:

- all `application` capabilities
- full `co.meta` support
- runtime reflection, instrumentation, dynamic loading, patching, and eval-based facilities

Internally forbidden:

- system and FFI capabilities unless reached through an imported public library surface
- raw pointers, addresses, and native functions in its own implementation

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`advanced` Libraries

Internally allowed:

- all ordinary safe language features
- macros and templates
- async, parallel, continuation, scheduling, and transformation machinery

- compiler/runtime behavior definitions permitted to advanced libraries

Internally forbidden:

- raw pointers
- references and addresses
- native functions

Public boundary:

- surface `struct` contracts
- approved built-in types
- deep-snapshot transfer

`system` Libraries

Internally allowed:

- pointers, references, addresses, and word-level types
- native functions
- structs and cstructs
- units containing free functions
- basic value-typed functions
- type aliases
- basic threading and process primitives

Internally forbidden:

- classes
- modules
- interfaces and signatures
- overloading and overriding
- new operator definitions
- generics and templates
- macros
- reflection and dynamic runtime
- dependent types and type classes
- function patterns
- closures and currying
- advanced concurrency machinery such as continuations, defer, lazy evaluation, and thunks
- variadic, named, optional, and default parameters

Public boundary:

- surface `cstruct` contracts
- ABI-safe built-in types
- system ABI value transfer

`ffi` Libraries

Internally allowed:

- native and extern declarations
- pointers, references, addresses, and C ABI types
- cstructs and restricted implementation structs
- units containing free functions
- foreign calling-convention and symbol-linkage metadata

Internally forbidden:

- classes
- modules

- interfaces and signatures
- overloading and overriding
- new operator definitions
- generics and templates
- macros
- reflection and dynamic runtime
- dependent types and type classes
- closures, currying, and advanced concurrency machinery

Public boundary:

- surface `cstruct` contracts
- C-ABI-safe built-in types
- C ABI value transfer

Dependency Direction

Allowed dependency flow is one-way:

```
application
    ↓ through public surface APIs only
dynamicvmrt
    ↓ through public surface APIs only
advanced
    ↓ through public surface APIs only
system
    ↓ through public surface APIs only
ffi
```

A library may depend on a library at the same or a lower level only when the dependency does not create a cycle. Reverse dependencies are compiler errors.

Cross-Library Communication

From	To	Allowed?	Notes
`application`	`dynamicvmrt`	✅	projected public surface only
`application`	`advanced`	✅	projected public surface only
`application`	`system`	✅	projected public surface only
`application`	`ffi`	✅	projected public surface only
`dynamicvmrt`	`advanced`	✅	projected public surface only
`dynamicvmrt`	`system`	✅	projected public surface only
`dynamicvmrt`	`ffi`	✅	projected public surface only
`dynamicvmrt`	`application`	❌	reverse dependency
`advanced`	`system`	✅	projected public surface only
`advanced`	`ffi`	✅	projected public surface only
`advanced`	`application`	❌	reverse dependency
`advanced`	`dynamicvmrt`	❌	reverse dependency
`system`	`ffi`	✅	projected public surface only
`system`	`application`	❌	reverse dependency
`system`	`dynamicvmrt`	❌	reverse dependency
`system`	`advanced`	❌	reverse dependency
`ffi`	`application`	❌	reverse dependency

From	To	Allowed?	Notes
`ffi`	`dynamicvmrt`	❌	reverse dependency
`ffi`	`advanced`	❌	reverse dependency
`ffi`	`system`	❌	reverse dependency

Units in detail

A `unit` is a named, non-instantiable container for functions.

Like other named primary declarations, a unit may use an explicit name or `_` for filename-derived naming. For example, `_ co.lang.unit` in `Math.fol` declares `Math`, while an explicit name such as `Math co.lang.unit` is authoritative regardless of the filename.

Its purpose is structural: it prevents functions from flowing freely at package-file scope and preserves FoLang's rule that every ordinary package source file has one primary top-level declaration. A unit does **not** require its functions to form a domain model, capability, service, or other semantic abstraction.

Functions within a unit may be strongly related by purpose—for example, mathematical, parsing, encoding, or validation functions—or only loosely related. Semantic cohesion is encouraged as a source-design practice but is not enforced by the language.

A unit has two forms:

1. **Standalone unit** — its name does not match a struct in the same package. It acts as an ordinary named container for receiverless functions.
2. **Struct companion unit** — its name matches exactly one `co.lang.struct` in the same package. It provides behaviour associated with that struct while the struct declaration itself remains pure data. For more details please refer section Struct Companion Units

Only `co.lang.struct` can have a companion unit. A same-name unit is not allowed for `co.lang.class`, `co.lang.cstruct`, modules, enums, unions, interfaces, signatures, or other declaration kinds. Classes already contain their own methods and lifecycle behaviour; `cstruct` remains a restricted C-compatible data representation.

```
Text co.lang.unit = {

    trim(value co.lang.string)->(co.lang.string) = {
        ...
    }

    contains(
        value  co.lang.string,
        search co.lang.string
    )->(co.lang.bool) = {
        ...
    }

    repeat(
        value co.lang.string,
        count co.lang.int
    )->(co.lang.string) = {
        ...
    }
}
```

Unit functions are accessed through the unit name:

```
cleaned := Text.trim(input);
found   := Text.contains(input, "FoLang");
```

A unit:

- introduces a named scope for its functions
- contains receiverless functions
- may additionally contain associated and operator functions when it is a struct companion unit

- requires the first declared parameter of every receiverless companion function to have the matching struct type
- uses the explicit receiver, rather than ordinary-parameter matching, to associate an associated function with its struct
- may contain public and private functions
- may use built-in types, user-defined types, or both
- does not introduce a user-defined data type or ADT
- has no fields or unit-level variables
- has no constructors or lifecycle methods
- has no instances, object identity, inheritance, or polymorphic dispatch
- cannot be instantiated or assigned as an object value
- cannot contain nested classes, structs, enums, modules, or other primary type declarations

Physical type or function declarations inside an individual unit function are not permitted. A separately declared class, struct, enum, module, or function may instead be restricted to one or more exact unit functions with `@co.dap.local`, using each function's complete qualified signature in the target set.

```
General co.lang.unit = {

    parseNumber(value co.lang.string)->(co.lang.int) = { ... }

    clamp(
        value co.lang.int,
        min   co.lang.int,
        max   co.lang.int
    )->(co.lang.int) = { ... }
}
```

The functions inside a unit do not need to use UDTs or ADTs. A unit is especially useful for small operations that consume built-in values, perform a computation, and return built-in values.

A unit may also represent a clearly related family of operations:

```
Math co.lang.unit = {

    abs(value co.lang.int)->(co.lang.int) = { ... }

    max(
        a co.lang.int,
        b co.lang.int
    )->(co.lang.int) = { ... }

    sqrt(value co.lang.float)->(co.lang.float) = { ... }
}
```

The common purpose of these functions makes the source easier to understand, but the compiler does not require or attempt to prove that all functions in a unit are semantically related.

CStructs

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
Point co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}

Rect co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

cstruct Rules

```
always passed by value — never by reference
simple memory layout — no metadata
can contain only simple types and other cstructs
cannot contain co.lang.struct                ❌  has metadata
cannot contain co.lang.string                ❌  heap allocated
cannot contain co.lang.dynamic               ❌  runtime type info
cannot contain classes                       ❌  vtable, metadata
cannot contain modules                       ❌
cannot contain any heap allocated type       ❌
cannot have methods
cannot have associated functions
cannot embed co.lang.struct
safe to cross direct ABI and zone boundaries

allowed field types:
    co.lang.int, co.lang.uint, co.lang.float  ✅  primitives
    co.lang.bool, co.lang.char, co.lang.byte  ✅  primitives
    co.lang.int->([N])                         ✅  fixed size arrays
    co.lang.cstruct                            ✅  other cstructs
```

Packed cstruct — no padding, exact memory layout

Used for hardware registers, binary protocols, exact memory mapped formats:

```
@co.dap.packed
Register co.lang.cstruct = {
    flags  co.lang.uint8;
    status co.lang.uint8;
    data   co.lang.uint16;
}
```

SIMD cstruct — aligned for vector operations

Used for math, graphics, signal processing:

```
@co.dap.simd(align=16)
Vec4 co.lang.cstruct = {
    x co.lang.float;
    y co.lang.float;
    z co.lang.float;
    w co.lang.float;
}
```

Both together

```
@co.dap.packed
@co.dap.simd(align=32)
AVXVec co.lang.cstruct = {
    data co.lang.float;
}
```

> `@co.dap.packed` and `@co.dap.simd` are specialisations of `co.lang.cstruct` — same rules, same zone boundary safety. They are not separate types.

Structs

```
myStruct co.lang.struct={
    field1 co.lang.int;
    field2 co.lang.string;
    field3 co.lang.bool;
}
```

Struct Rules

```
structs cannot extend other structs
structs cannot contain methods directly
structs can compose other structs
structs cannot have default values to fields/members
structs can embed other structs (Go lang like)
structs can have a same-package companion unit with the same name
```

The struct declaration remains pure data. All behaviour associated with a struct is declared separately in its matching `co.lang.unit`.

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
E co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

// ✅ No conflict — id and name promoted as B's own fields
B co.lang.struct = {
    age co.lang.float;
    E;                   // embedded — id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" };   // all fields at same level
b.id    // direct — no b.E.id needed
b.name  // direct — no b.E.name needed
b.age   // direct
```

```
// ❌ Compiler error — name conflict between B.name and E.name
B co.lang.struct = {
    name co.lang.string;   // conflicts with E.name
    E;
    age  co.lang.float;
}
// Fix 1 — rename B's conflicting field
// Fix 2 — use explicit composition instead: e E;
```

```
// Explicit composition — no promotion, always qualified access
B co.lang.struct = {
    name co.lang.string;
    e    E;              // named field — no conflict, no promotion
    age  co.lang.float;
}

b.name    // B's own name
b.e.id    // E's id — always explicit
b.e.name  // E's name — always explicit
```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field
Embedded field, name conflict with outer	❌ Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	❌ Compiler error
Explicit composition (`e E`)	No promotion — always accessed via `b.e.field`

> FoLang does **not** silently shadow conflicting fields. Any name conflict is a compiler error — the programmer must make a conscious decision to rename or switch to explicit composition.

Struct Declaration Relationships

A struct cannot physically declare another struct, enum, class, module, function, signature, interface, or other named declaration inside its body. A struct body remains a pure data declaration containing fields and permitted embeddings only.

Use one of the following instead:

- **composition** through a named field
- **embedding** where the declaration kind's embedding rules permit it
- a separately declared target-local declaration restricted with `@co.dap.local`

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeAddress co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// Employee.fol
Employee co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address EmployeeAddress; // composition
}
```

The following physical nesting is invalid:

```
Employee co.lang.struct = {
    Address co.lang.struct = { // ❌ nested declaration
        city co.lang.string;
    }

    address Address;
}
```

```
structs cannot declare inner structs     ❌  compiler error — only through @co.dap.local
structs can declare inner enums/ADTs     ❌  compiler error — only through @co.dap.local or @co.dap.nested
structs cannot declare inner classes     ❌  compiler error — struct is pure data
structs cannot declare inner modules     ❌  compiler error — struct is pure data
```

`@co.dap.local` controls declaration visibility only. It does not automatically compose or embed the annotated declaration into the target struct.

Struct Companion Units

When a unit has the same name as a struct in the same package, it is the companion unit of that struct.

The struct and its companion unit are separate primary declarations and therefore normally reside in separate source files within the same package. They are shown together below only to illustrate their relationship.

```
// Vector.fol
Vector co.lang.struct = {
    x co.lang.float;
    y co.lang.float;
}

// Vector.unit.fol
_ co.lang.unit = {

    distance(
        left  Vector,
        right Vector
    )->(co.lang.float) = {
        ...
    }

    isZero(value Vector)->(co.lang.bool) = {
        this.return value.x == 0.0 && value.y == 0.0;
    }
}
```

Receiverless functions in a companion unit are accessed through the shared struct/unit name:

```
d := Vector.distance(first, second);
zero := Vector.isZero(value);
```

These functions are analogous to **static functions of a class** in call form, but FoLang additionally requires their **first declared parameter** to have the matching struct type. The first parameter establishes ownership by the companion unit. A matching struct type in a later parameter is insufficient.

```
Vector co.lang.unit = {
    distance(left Vector, right Vector)->(co.lang.float) = { ... } // ✅
    convert(value Vector, radix co.lang.int)->(co.lang.string) = { ... } // ✅

    invalid(scale co.lang.float, value Vector)->(Vector) = { ... }
    // ❌ first parameter is not Vector

    zero()->(Vector) = { ... }
    // ❌ no first parameter identifies Vector ownership
}
```

Therefore, factory-style receiverless functions such as `Vector.create(x, y)` and zero-parameter receiverless functions such as `Vector.zero()` are not valid under this rule because no ordinary parameter establishes ownership. A factory may instead be declared as a type-associated function, described below, or placed in a standalone unit with a different name.

Companion-unit functions do not make the struct a class and do not introduce object identity, inheritance, virtual dispatch, lifecycle methods, or unit-level state.

Associated Functions in a Companion Unit

A companion unit may contain instance-associated and type-associated functions. Both forms use an explicit receiver to establish association with the matching struct, but they differ in whether the receiver is a struct value or the struct type itself.

Instance-Associated Functions

An instance-associated function has an explicit receiver value whose type is the matching struct:

```
Vector co.lang.unit = {

    (value Vector) magnitude()->(co.lang.float) = {
        this.return co.math.sqrt(
            value.x * value.x + value.y * value.y
        );
    }

    (value Vector) scale(factor co.lang.float)->(Vector) = {
        this.return Vector{
            x: value.x * factor,
            y: value.y * factor
        };
    }
}
```

Associated functions may be invoked using method-call syntax:

```
v Vector=Vector{}
length := v.magnitude();
scaled := v.scale(2.0);
```

The method-call form is syntactic association. Conceptually:

```
v.magnitude()
```

resolves to the associated function declared in `Vector co.lang.unit` with `v` supplied as its explicit receiver. Associated functions are therefore analogous to **instance methods**, but they remain externally declared functions and do not acquire class semantics.

Instance-associated-function rules:

- the explicit receiver value must have the struct type whose name matches the unit name
- the receiver establishes association; the ordinary parameter list does not need to contain the matching struct type
- the matching struct and unit must be declared in the same package
- the function cannot be declared loose at package scope
- it has no inheritance, overriding, or virtual dispatch
- it receives no special private access beyond normal visibility rules
- an imported struct with the same short name does not create a companion relationship

```
Employee co.lang.struct = {
    id co.lang.int;
}

Employee co.lang.unit = {
    (emp Employee) isValid()->(co.lang.bool) = { ... }       // ✅
    (dept Department) isValid()->(co.lang.bool) = { ... }   // ❌ receiver does not match Employee
}
```

Type-Associated Functions

A type-associated function uses the matching struct type itself as its explicit receiver. It is analogous to a class-level or static factory function in call form, but it does not introduce class lifecycle, inheritance, or object-oriented dispatch.

```
Vector co.lang.unit = {
    (Vector) zero()->(Vector) = {
        this.return Vector{x: 0.0, y: 0.0};
    }

    (Vector) create(
        x co.lang.float,
        y co.lang.float
    )->(Vector) = {
        this.return Vector{x: x, y: y};
    }
}
```

Type-associated functions are invoked through the struct name:

```
origin := Vector.zero();
value  := Vector.create(10.0, 20.0);
```

Type-associated-function rules:

- the explicit receiver must be the struct type whose name matches the unit name
- the type receiver establishes association, so no ordinary parameter is required to have the matching struct type
- zero ordinary parameters are permitted
- the matching struct and unit must be declared in the same package
- the function cannot be declared loose at package scope
- the struct type is not an object instance and cannot be mutated through the type receiver
- type association does not add constructors, lifecycle methods, inheritance, overriding, or virtual dispatch
- an imported struct with the same short name does not create a companion relationship

The three companion-function forms are therefore:

```
receiverless companion function
    -> first ordinary parameter struct
    -> parameter not matching struct (as companion unit name match and must match with struct name)

instance-associated function
    -> explicit receiver is a value of the matching struct

type-associated function
    -> explicit receiver is the matching struct type itself
```

Companion Name Rules

Within one package, FoLang may contain:

```
one Vector co.lang.struct
one Vector co.lang.unit
```

Together they form one struct/companion pair. The following are compiler errors:

```
two units named Vector in the same package
a unit matching a class or cstruct
a companion unit located in a different package from its struct
an instance-associated function whose receiver value is not of the matching struct type
a type-associated function whose type receiver is not the matching struct type
```

Name resolution is determined by context:

```
v Vector;                // type position → Vector struct
v := Vector{x: 1, y: 2};  // construction → Vector struct
z := Vector.isZero(v);    // qualified function lookup → Vector companion unit
m := v.magnitude();       // associated-function lookup → Vector companion unit
```

```
x General = General{};  // ❌ compiler error — a unit is not instantiable
```

Companion Function Categories and Operator Functions

```
Employee co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

Employee co.lang.unit = {

    // Receiverless companion function: first parameter establishes ownership.
    compare(
        left  Employee,
        right Employee
    )->(co.lang.int) = {
        ...
    }
    // Recieverless companion function: without matching first parameter with type
    getEmployee(id co.lang.string)->(Employee)={...}


    // Instance-associated function: receiver is an Employee value.
    (emp Employee) fetchEmployee(
        empId co.lang.string
    )->(Employee) = {
        ...
    }

    // Type-associated function: receiver is the Employee type.
    (Employee) getInstance()->(Employee) = {
        ...
    }
}
```

Call forms:

```
order  := Employee.compare(emp, other); // receiverless companion function
result := emp.fetchEmployee("E1");      // instance-associated function
emp    := Employee.getInstance();       // type-associated function
emp1   :=Employee.getEmployee("E0021");   // receiverless companion function
```

The receiver remains explicit in associated-function declarations even though instance-call or type-call syntax is available at the call site. This does not give the struct class semantics: there is no inheritance, overriding, virtual dispatch, hidden receiver, or lifecycle.

Operator functions associated with a struct must be declared in its companion unit. They may use either an instance receiver or a type receiver:

```
Employee co.lang.unit = {

    @co.dap.operator(symbol="+")
    (emp Employee) add(other Employee)->(Employee) = {
        ...
    }

    @co.dap.operator(symbol="==")
    (Employee) equals(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        ...
    }
}
```

A receiverless operator function whose first parameter is the matching struct is accepted as syntactic shorthand for the equivalent type-associated operator form. The compiler normalizes both forms to the same companion ownership model; declaring both equivalent forms for the same operator signature is a duplicate-definition error.

> Exception from normal associate functions, operator functions cannot have receiverless functions with non matching first parameter to type (struct). otherwise it is a compiler error.

Unions

Unions are untagged ADTs

```
 myUnion co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Enums

```
myEnum co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

classes

```
Employee co.lang.class ={
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating — internally redirecting the call to module function
}

// $1, $2, $3 ... are previous results captured as bind variables
Emp co.lang.class={
    dosomething(a co.lang.int, b co.lang.int)->
(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

Class Declaration Relationships

A class cannot physically contain named class, struct, enum, module, function, signature, interface, or other declaration definitions as nested declarations. Class bodies contain fields, lifecycle declarations, and methods permitted by the class model.

Types and helper declarations used only by one class are declared in their ordinary legal source locations and restricted to that class with `@co.dap.local`:

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeAddress co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// EmployeeStatus.fol
@co.dap.local(for=hr.employee.Employee)
EmployeeStatus co.lang.enum = {
    Active,
    Inactive,
    Pending
}
```

```
// Employee.fol
Employee co.lang.class = {
    address EmployeeAddress;
    status  EmployeeStatus;

    getAddress()->(EmployeeAddress) = {
        this.return this.address;
    }
}
```

The local declarations are visible while compiling `hr.employee.Employee`, but they do not become nested names such as `Employee.EmployeeAddress` or `Employee.EmployeeStatus`.

The following is invalid:

```
Employee co.lang.class = {
    Address co.lang.struct = { // ❌ physical nested declaration
        city co.lang.string;
    }
}
```

Ordinary visibility annotations do not widen a target-local declaration beyond the declaration or closed target set named by `@co.dap.local` or `@co.dap.nested`.

Method Types

```
Employee co.lang.class ={

    @co.dap.static
    getEmployee()->(Employee) ={}

    @co.dap.instance
    getEmployee()->(Employee)={}

    @co.dap.class
    getEmployee()->(Employee) ={}

    @co.dap.object
    getEmployee()->(Employee)={}
}

@co.dap.oops(
    A: { inherit:true, virtual:true },
    B: { implements:true },
    C: { inherits:true, abstract=true },
    D: { inherits:true },
    E: { uses:true },
    F: { composes:true },
    G: { extends:true },
    H: { with:true },
    I: {assiociate:true},
)
test co.lang.class ={
    getTest(id int)->(test) ={}
}
```

The @@new and @@init Methods

`@@new` and `@@init` are lifecycle methods — compiler-owned, not user-definable outside the class. `@@` signals they are restricted lifecycle symbols, not regular methods.

```
@co.dap.generic(type={T:{typename}, R:{typename}})
Employee co.lang.class = {

    id T
    name R

    // @@new is provided by default even if not overridden.
    // Override only when you genuinely need to change allocation behavior.

    @co.dap.method.class
    @co.dap.private
    @@new()->(co.lang.uninit) = { self.return co.const.none }

    @co.dap.method.class
    @co.dap.public
    @@new(a co.lang.typevalue, b co.lang.typevalue)->(co.lang.uninit) = {
        // Manual type aliasing — @co.dap.generic handles this automatically
        // Override @@new only when you need custom allocation logic
        T co.lang.type = a
        R co.lang.type = b

        // self keyword is allowed only in class methods
        self.parent.new();

        // uninit instance method internally calls new and init
        self.return co.lang.uninit.instance(Employee, self);
    }

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee) = {}
}
```

Anonymous Classes/Types

```
emp := co.lang.class{};

empObj := emp.init();

empobj1 := co.lang.class{
    name string
}.init();
```

Interfaces

```
IEmployee co.lang.interface = {
    storeEmployee(emp Employee)->(Employee);
}
```

Signatures

```
// Employee is an ordinary package-level declaration.
MEmployee co.lang.signature = {
    storeEmployee(emp Employee)->(Employee);
}
```

Structurally they look similar — both are lists of contracts. The difference is **who implements them and how**.

	`co.lang.signature`	`co.lang.interface`
Implemented by	module via `matches=`	class via `implements=[]`
Number of implementations	Any number of distinct modules may match one signature	Any number of classes may implement one interface
Runtime cardinality of one implementation declaration	One module object per loaded module identity	Any number of independently constructed class objects
State model	One shared state for all references to the same module	Separate per-instance state for each class object
May specify required values	✅	❌ — methods only
May reference package-level types	✅	✅
May declare abstract/fixed type components	✅	❌
May require generic type constructors	✅	❌
May declare physical nested/local types	❌	❌
May use `@co.dap.local`	❌	❌
Instantiation involved	Module is declared once, not constructed	Class objects are constructed
Reference use	Compatible module references may be used through the signature type without creating another module	Interface references may refer to any implementing object instance
OOP dispatch	❌	✅ virtual/dynamic
Contract style	module values, functions, and type components	behavioral methods on object instances
Practical analogy	singleton component contract	object-instance behavioral contract
Origin	ML/OCaml-inspired modules	Java/C#/Go interfaces

- A `signature` is a **module contract** over values, functions, existing package-level types, and abstract or fixed type components. A type-component specification is a contract slot, not a physical nested type definition. Multiple modules may match one signature, but each module declaration denotes one module object with shared module state.
- An `interface` is a **behavioral contract** tied to class dispatch and polymorphism. It cannot declare module type components or own nested type definitions. A class implementing an interface may create any number of independent runtime objects.
- The approximation `module + signature ≈ singleton object + interface` is useful for understanding cardinality and shared state, but a module is a language-level component rather than a class-based singleton pattern.

Modules

A module is an ML/OCaml-style abstraction governed by an optional signature. A module may use package-level types and may satisfy type components declared by its signature, but it does not physically own or nest arbitrary type declarations. A module should not be introduced merely to prevent functions from appearing loose in a file; use `co.lang.unit` for that simpler structural purpose.

```
// Employee.fol — ordinary package-level type
Employee co.lang.struct = {
    Id   co.lang.int;
    Name co.lang.string;
}

// EmployeeModule.signature.fol
EmployeeModule co.lang.signature = {
    getEmployee(id co.lang.int)->(Employee);
}

// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
EmployeeModImpl co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

    getEmployee(id co.lang.int)->(Employee) = {
        this.return Employee{
            Id: 10,
            Name: "Rao"
        };
    }
}

mm EmployeeModule = EmployeeModImpl;
v Employee = mm.getEmployee(10);
```

Module Cardinality and Singleton Analogy

A module declaration defines exactly one named module object for that loaded module identity. It is not an instantiable blueprint and does not create a new module object each time its name is referenced.

```
first  EmployeeModule = EmployeeModImpl;
second EmployeeModule = EmployeeModImpl;
```

`first`, `second`, and `EmployeeModImpl` refer to the same module object. These bindings copy or retain the module reference; they do not clone or instantiate the module. Consequently, values declared directly by the module represent one shared module state for all references to that module.

A signature does not itself create a module object. It is a contract, and any number of separately declared modules may conform to the same signature:

```
DatabaseBackend signature
├── PostgreSQLBackend module   -> one named module object
├── MySQLBackend module        -> one named module object
└── TestDatabaseBackend module -> one named module object
```

Each conforming module declaration contributes its own single module object and its own module state. The signature does not restrict the number of distinct conforming module declarations.

A useful analogy is:

```
signature          ≈ interface contract
conforming module  ≈ singleton object implementing that contract
class              ≈ instantiable object type
```

The analogy is intentionally limited. A FoLang module is a compiler-recognized named component, not a class made singleton through a private constructor, static field, or runtime pattern. It cannot be repeatedly constructed. Because module references are first-class in FoLang, they may be bound and used through a compatible signature type, but every reference to the same module declaration still denotes the same module object.

Modules are also broader than ordinary interface implementations. A matching module may provide module values, functions, and abstract, fixed, or generic type-component bindings required by its signature. An interface constrains object behaviour; it does not provide the same module type-component abstraction.

By contrast, a class declaration defines an instantiable type. Every class construction creates a distinct object with independent identity, state, and lifetime:

```
PostgreSQLBackend module
└── one shared module object and module state

PostgreSQLConnection class
├── connection1 -> independent object and state
├── connection2 -> independent object and state
└── connection3 -> independent object and state
```

> **Formal mental model:** A FoLang module is a single named implementation component that may conform to a signature. It is comparable to a singleton object implementing an interface, but it is not instantiated from a class. Multiple distinct modules may conform to the same signature, while each module declaration denotes one module object. Unlike an ordinary singleton-interface implementation, a module may also satisfy abstract, fixed, and generic type components required by its signature.

> **Module instantiation** A FoLang class or struct declaration introduces an instantiable type but does not create an instance. A FoLang module declaration introduces one named module component directly into its package. The module name acts as the binding for that component, so no separate construction expression is required. The module's runtime state is initialized once according to the language's module-initialization rules.

> A module declaration is a singleton component declaration and binding, rather than merely an object definition.

Module Signature Contents

A `co.lang.signature` is a declarative contract for a module. It may specify required module values, functions, and type components. A signature does not allocate storage, initialize variables, execute statements, or provide function bodies.

A signature may contain:

- value specifications
- function signatures
- references to already existing accessible types
- abstract type-component specifications
- fixed or manifest type-component specifications
- abstract generic type-constructor specifications

A signature may not contain:

- value initializers
- executable statements
- function bodies
- concrete class, struct, enum, module, interface, or signature definitions
- arbitrary nested or target-local declarations

Type-component specifications are part of module conformance. They are not Java-, C++-, or C#-style inner types and do not participate in `@co.dap.local`.

Value Specifications

A declaration such as:

```
Counter co.lang.signature = {
    count co.lang.int;
    increment(amount co.lang.int)->();
}
```

requires a matching module to provide a value named `count` of type `co.lang.int` and a compatible `increment` function. The signature does not initialize `count` and does not define `co.lang.int`; the built-in type already exists.

```
CounterImpl co.lang.module->(
    signature=Counter,
    matches=Counter
) = {
    count co.lang.int = 0;

    increment(amount co.lang.int)->() = {
        count.value = count + amount;
    }
}
```

The same rule applies when a value or function specification uses an existing accessible package type:

```
EmployeeRepository co.lang.signature = {
    current hr.employee.Employee;
    find(id co.lang.int)->(hr.employee.Employee);
}
```

The matching module must provide `current` and `find`. It does not redefine `hr.employee.Employee`.

Abstract Type Components

An abstract type component declares that every matching module must supply a type binding for that component:

```
Repository co.lang.signature = {
    Entity co.lang.type;

    current Entity;
    find(id co.lang.int)->(Entity);
}
```

`Entity co.lang.type;` does not define the representation of `Entity`. It defines a required module type component. A matching module binds it to a compatible existing type:

```
EmployeeRepositoryImpl co.lang.module->(
    signature=Repository,
    matches=Repository
) = {
    Entity co.lang.type = hr.employee.Employee;

    current Entity = ...;
    find(id co.lang.int)->(Entity) = { ... }
}
```

Within a matching module, `Entity co.lang.type = ...` is a **signature-component binding**, not an arbitrary nested type declaration. Its name must correspond to an abstract type component declared by the matched signature. A module cannot use this form to introduce unrelated module-local types.

An abstract type component differs from `forward` and `extern` declarations:

```
abstract signature type component
    -> each matching module supplies its own compatible type binding

forward declaration
    -> one specific declaration is completed later

extern declaration
    -> one specific declaration is defined in another linkage or compilation unit
```

Fixed or Manifest Type Components

A signature may fix a type component to an already known type:

```
IntegerRepository co.lang.signature = {
    Id co.lang.type = co.lang.int;

    find(id Id)->(co.lang.bool);
}
```

Here `Id` is predetermined as `co.lang.int`. A matching module uses that type component but does not choose or redefine it.

```
Entity co.lang.type;               -> abstract; implementor supplies the binding
Id co.lang.type = co.lang.int;     -> fixed; signature supplies the type equality
```

Abstract Generic Type Constructors

A signature may require a generic type constructor without defining its representation:

```
StackSignature co.lang.signature = {
    Stack(T) co.lang.type;

    empty(T)->(Stack(T));
    push(value T, stack Stack(T))->(Stack(T));
    pop(stack Stack(T))->(T, Stack(T));
}
```

`Stack(T) co.lang.type;` declares an **abstract generic type component**, also described as an abstract type constructor of arity one. The signature specifies that `Stack` accepts one type argument, but it does not define what `Stack(T)` is.

A matching module must provide a compatible type-constructor binding with the same name, arity, and declared constraints:

```
ListStackModule co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.type = co.core.list(T);

    empty(T)->(Stack(T)) = { ... }
    push(value T, stack Stack(T))->(Stack(T)) = { ... }
    pop(stack Stack(T))->(T, Stack(T)) = { ... }
}
```

Another matching module may choose another representation:

```
ArrayStackModule co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.type = collections.ArrayStack(T);
    ...
}
```

Therefore:

```
StackSignature
    -> requires a generic type constructor Stack(T)

ListStackModule
    -> binds Stack(T) to co.core.list(T)

ArrayStackModule
    -> binds Stack(T) to collections.ArrayStack(T)
```

A signature-component binding does not permit physical type nesting. When an implementation needs a new concrete struct, class, or enum representation, that declaration remains an ordinary package declaration and may be restricted to the implementing module with `@co.dap.local`; the module then binds the signature component to that declaration.

Signature Conformance Rules for Type Components

For every type component in a matched signature:

- an abstract component must receive exactly one compatible module binding
- a fixed component must retain the type equality declared by the signature
- a generic component binding must preserve generic arity, parameter kinds, bounds, variance, and other declared constraints
- component names must be unique within the signature
- component bindings cannot contain executable code
- extra module-local type bindings that do not correspond to signature components are compiler errors
- types referenced by value and function specifications must resolve after applying the module's type-component bindings

Module Declaration Relationships

A module cannot physically declare nested structs, enums, classes, modules, signatures, interfaces, or other arbitrary named declarations. It references ordinary package-level declarations through its functions and signature. The only type-like declarations permitted directly in a matching module are signature-component bindings that satisfy abstract type components declared by its matched signature; such bindings do not create independent nested declarations.

A declaration intended only for one module may be restricted with `@co.dap.local`:

```
// EmployeeModuleConfig.fol
@co.dap.local(for=hr.employee.EmployeeModImpl)
EmployeeModuleConfig co.lang.struct = {
    timeout co.lang.int;
    retries co.lang.int;
}
```

```
// EmployeeModImpl.fol
EmployeeModImpl co.lang.module = {
    connect(cfg EmployeeModuleConfig)->(co.lang.bool) = {
        ...
    }
}
```

The following remains invalid:

```
EmployeeModImpl co.lang.module = {
    Config co.lang.struct = { // ❌ physical nested declaration
        timeout co.lang.int;
    }
}
```

```
A target-local declaration does not automatically become a module member name and is not projected through the
module's signature. It becomes part of the signature view only when an explicit signature type component is bound
to it or a signature value/function specification references it through an allowed type component.
```

Structs vs Classes vs Modules vs Units vs Packages

	Struct	CStruct	Class	Module	Unit	Package
Purpose	Pure data shape	C-like value type	Behaviour + data	Signature-backed ML-style abstraction	Named function container; optionally a struct companion	Folder-based grouping
Fields	✅	✅ simple only	✅ per instance	❌	❌	❌
Module-level values	❌	❌	❌	✅ when declared directly or required by a signature	❌	❌

	Struct	CStruct	Class	Module	Unit	Package
Functions / methods	Optional static-like functions whose first parameter is the struct, plus receiver-based associated functions, through a matching companion unit	❌	✅ methods	✅ module functions	✅ receiverless functions; associated functions only when matching a struct	❌
Lifecycle (`@@new` / `@@init`)	❌	❌	✅	❌	❌	❌
`this` / `self`	❌	❌	✅	❌	❌	❌
Value/literal construction	✅	✅	❌	❌	❌	❌
Lifecycle instantiation (`new` / `init`)	❌	❌	✅ multiple objects	❌ — one module object per declaration	❌	❌
Runtime state cardinality	Per bound struct object	Per value	Per class object	One shared state for the module declaration	—	—
First class	✅	✅	✅	✅ module reference; referencing does not instantiate	❌	❌
Pass by	Reference	Value	Reference	Reference to the same module object	—	—
Contract	—	—	`interface` via `implements=[]`	`signature` via `matches=`	none	—
OOP / inheritance	❌	❌	✅	❌	❌	❌
Physically nested declarations	❌	❌	❌	❌	❌	✅ separate declarations across package files
May be an `@co.dap.local` target	✅	❌	✅	✅	❌	❌
Pattern matching	✅	✅	✅	❌	❌	❌
Direct ABI / zone boundary safe	❌ — library boundaries require snapshots	✅	❌	❌	❌	❌
Associated functions	✅ through matching companion unit	❌	—	—	✅ only in a struct companion unit	❌
Embedding	✅	❌	—	—	❌	❌
Declared with	`co.lang.struct`	`co.lang.cstruct`	`co.lang.class`	`co.lang.module`	`co.lang.unit`	folder path
C++ backend analogy	struct without methods	plain C struct	class	struct/class abstraction	namespace or static function scope	namespace

	Struct	CStruct	Class	Module	Unit	Package
Closest mental model	Rust struct	C struct	Java/C# class	singleton implementation component with ML-style type members	named function scope; optional struct companion	filesystem namespace

Mental model:

```
reach for struct   → pure data; use a same-name companion unit for behaviour
reach for cstruct  → physical ABI-compatible value data crossing direct zone or native boundaries
reach for class    → behaviour, lifecycle, multiple instances
reach for module   → one named implementation component with shared state, governed by an optional signature and
capable of satisfying type components
reach for unit     → named function container; same-name struct unit acts as companion
reach for package  → folder-based grouping only, not a value
```

> **Declaration scoping rule:** FoLang does not permit physical nested or function-local named declarations. Classes, structs, enums, modules, and functions use ordinary package declarations and may restrict a supported declaration to one or more exact targets with `@co.dap.local`. The annotation accepts either one declaration reference or a non-empty closed target list. The local declaration and every target must belong to the same exact folder-derived package; parent and subpackages are different packages. Non-function targets are identified by complete qualified name; overloaded function targets are identified by complete qualified signature. Visibility is the union of the explicitly listed target scopes and is neither inherited nor transitive. Signatures and interfaces cannot own or target local declarations. A signature may nevertheless declare abstract, fixed, and generic type components as module-conformance requirements; these are contract slots rather than physical nested declarations. A matching module may bind only those declared components. Units contain functions only, and a struct companion unit remains a separate declaration from its struct.

Local and/or Nested types and functions

FoLang does not provide Java-, C++-, or C#-style physical nesting of named declarations inside another type, module, function, or executable block. Instead, FoLang keeps declarations in their ordinary legal source locations and may restrict one declaration to one or more explicitly identified target declarations.

```
@co.dap.local(for=<declaration-reference>)
```

or:

```
@co.dap.local(
    for=[
        <declaration-reference>,
        <declaration-reference>
    ]
)
```

The annotated declaration is called a **target-local declaration**. The declarations named by `for` form its **local target set**.

Supported Declaration and Target Kinds

`@co.dap.local` may be applied only to these declaration kinds:

- `co.lang.class`
- `co.lang.struct`
- `co.lang.enum`
- `co.lang.module`
- a named function declared in a context that normally permits functions

Every entry in the local target set must resolve to exactly one:

- class
- struct

- enum
- module
- function overload

A target list may contain any combination of supported target kinds.

Signatures and interfaces do not support target-local or physically nested declarations. A `co.lang.signature` or `co.lang.interface` cannot be annotated with `@co.dap.local` and cannot appear in a local target set. A signature may declare abstract, fixed, or generic **type-component specifications** as part of its module contract; these are contract requirements rather than local or nested type definitions. Interfaces do not declare signature type components.

Other declaration kinds are not target-local unless a later section explicitly permits them.

Target-Set Syntax

The `for` argument accepts either:

1. one declaration reference; or
2. a non-empty list of declaration references.

The scalar form is equivalent to a singleton target list:

```
@co.dap.local(for=hr.employee.Employee)
```

```
@co.dap.local(for=[hr.employee.Employee])
```

The scalar form is canonical when only one target is required. Use the list form when two or more declarations require access:

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService,
        hr.employee.EmployeeValidator
    ]
)
EmployeeState co.lang.enum = {
    Active,
    Inactive
}
```

Rules:

- the target list must contain at least one declaration reference
- each reference must resolve independently and unambiguously
- duplicate references to the same resolved declaration are a compiler error
- list order does not affect visibility or declaration identity
- the target list is a closed set; access is not granted to declarations omitted from it
- repeated `@co.dap.local` annotations are not used to accumulate targets; use one annotation with one complete target list

Invalid:

```
@co.dap.local(for=[])
State co.lang.struct = { ... }
// ❌ empty target list
```

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.Employee
    ]
)
State co.lang.struct = { ... }
// ❌ duplicate resolved target
```

Declaration-Reference Syntax

Each `for` entry is a compiler-resolved declaration reference, not a string, runtime expression, function call, or `co.meta.symbol` value.

For a non-function target, use only its complete qualified declaration name:

```
@co.dap.local(for=hr.employee.Employee)
@co.dap.local(for=hr.employee.EmployeeStatus)
@co.dap.local(for=hr.employee.EmployeeRules)
```

For a function target, use its complete qualified function signature because FoLang permits overloads:

```
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.decimal)->()
)
```

Function references in a target list follow the same rule:

```
@co.dap.local(
    for=[
        hr.employee.Employee.calculate(co.lang.decimal)->(),
        hr.employee.Employee.calculate(co.lang.int)->(),
        hr.employee.Employee.validate()->(co.lang.bool)
    ]
)
CalculationState co.lang.struct = { ... }
```

Parameter names are not part of the reference:

```
// ✅ canonical
hr.employee.Employee.calculate(co.lang.decimal)->()

// ❌ parameter names are not declaration identity
hr.employee.Employee.calculate(amount co.lang.decimal)->()
```

The parameter and return types must match one exact overload. An abbreviated function name, unresolved target, or ambiguous overload is a compiler error.

```
@co.dap.local(for=hr.employee.Employee.calculate)
// ❌ ambiguous when calculate is overloaded
```

Each listed overload is an independent target. Listing one overload does not grant access to sibling overloads with the same function name.

Same-Package Requirement

The target-local declaration and **every declaration in its local target set** must belong to the same exact package. The compiler compares their complete folder-derived package identities; matching only a parent package, package family, import alias, realm, library, or source root is not sufficient.

For a function target, the target package is the package that owns the function's enclosing class, struct companion unit, module, unit, or other legal function container. A target-local function is checked in the same way: the package containing its legal function-owning declaration must match the package of every listed target.

The invariant is:

```
package(target-local declaration)
    == package(target 1)
    == package(target 2)
    == ...
```

```
// hr/employee/Employee.fol
Employee co.lang.class = { ... }

// hr/employee/EmployeeService.fol
EmployeeService co.lang.class = { ... }

// hr/employee/EmployeeState.fol
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
// ✅ all declarations belong to package hr.employee
```

A declaration in another package cannot participate in the local target set, even when ordinary package visibility, imports, aliases, parent/subpackage relationships, or library membership would otherwise make the declaration resolvable.

```
// EmployeeState is declared in package hr.internal
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.internal.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
// ❌ local declaration and every target must have the same package
```

```
// CalculationState is declared in package hr.employee.internal
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.decimal)->()
)
CalculationState co.lang.struct = { ... }
// ❌ subpackages are distinct packages; both sides must be exactly hr.employee
```

The same-package rule prevents `@co.dap.local` from becoming a cross-package friendship or visibility-bypass mechanism. A local declaration is a selectively shared implementation detail inside one package, not an imported helper attached from elsewhere.

Visibility Domain

A target-local declaration may be resolved only from:

1. each exact declaration in its local target set; and
2. lexical scopes contained within each listed target's implementation.

For targets `A`, `B`, and `C`, the visibility domain is the union of their implementation scopes:

```
visibility(local declaration)
    = scope(A) ∪ scope(B) ∪ scope(C)
```

It is not an intersection, and code does not need to be simultaneously inside every target.

Consequences:

- a declaration local to a class is available to that class body and its methods
- a declaration local to a module is available to that module body and its functions
- a declaration local to a struct is available while resolving that struct declaration
- a declaration local to an enum is available while resolving that enum declaration
- a declaration local to a function is available only in the body of the exact targeted overload and its nested statement scopes
- when several targets are listed, each listed target receives access independently
- sibling declarations and sibling overloads not listed cannot resolve it
- subclasses, companion units, extensions, callees, callers, and related declarations do not gain access automatically
- visibility is not inherited, transitive, or propagated through calls, composition, embedding, imports, or other local declarations

- it cannot be imported, exported, or projected through a library surface

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = {
    Active,
    Inactive
}

Employee co.lang.class = {
    state EmployeeState; // ✅ listed target
}

EmployeeService co.lang.class = {
    state EmployeeState; // ✅ listed target
}

Payroll co.lang.class = {
    state EmployeeState; // ❌ not listed
}
```

Interaction with Ordinary Visibility

`@co.dap.local` establishes the final visibility boundary. Ordinary visibility annotations such as `@co.dap.public`, `@co.dap.package`, `@co.dap.protected`, or `@co.dap.private` cannot widen the declaration beyond its closed local target set.

```
@co.dap.public
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
EmployeeState co.lang.enum = { Active, Inactive }
```

`EmployeeState` is still visible only to the listed targets. The ordinary visibility annotation is redundant for external resolution and may produce a compiler warning, but it never overrides `@co.dap.local`.

When most or all declarations in a package require access, ordinary package visibility should be used instead of maintaining a large target list.

Source Placement and Identity

A target-local declaration remains in the source position normally required for its declaration kind:

- a class, struct, enum, or module remains a normal primary declaration and follows the one-primary-declaration-per-package-file rule
- a function remains inside a unit, class, module, or another declaration context that normally permits functions
- `@co.dap.local` does not permit a free function to appear loose at package-file scope

The declaration retains a stable package-owned compiler identity. Its package identity must be exactly the same as the package identity of every target. It does not become physically nested and is not addressed as `Target.LocalName`.

```
hr.employee.EmployeeState
    local-for [
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
```

The normalized local target set is declaration metadata. Target-list order does not create a different declaration identity.

No Implicit Relationship or Privilege

`@co.dap.local` changes visibility only. It does not imply:

- composition
- embedding
- inheritance
- lifecycle ownership
- memory ownership
- friendship or privileged private-member access
- automatic membership in any listed target
- shared runtime state among the targets

Composition must still be written explicitly through a field or another supported relation. Embedding remains a separate facility: only struct and enum declarations are eligible for embedding, and each use must follow the embedding rules defined for that declaration kind.

Escape Restrictions

A target-local type must not leak outside its complete visibility domain.

It cannot appear in:

- a public, package, or protected API visible outside the local target set
- a library surface
- the parameter or return types of a function to which it is local
- a field or method signature that exposes it beyond the local target set
- an exported generic specialization or type alias

A value whose static type is target-local must be converted to an externally visible type before leaving the union of the listed targets' visibility domains.

Invalid Physical Nesting

```
Employee co.lang.class = {
    Address co.lang.struct = { ... } // ❌
}

EmployeeModule co.lang.module = {
    Config co.lang.struct = { ... } // ❌
}

process()->() = {
    State co.lang.enum = { Ready, Done } // ❌
}

EmployeeContract co.lang.signature = {
    Employee co.lang.struct; // ❌
}

EmployeeApi co.lang.interface = {
    Result co.lang.struct; // ❌
}
```

Use separately declared package declarations, composition, embedding where allowed, and `@co.dap.local` when a closed set of declarations requires selective access.

There is another annotation `@co.dap.nested` which is similar to local but captures target state.

@co.dap.nested(target=hr.emp.Employee)

Instead of `for` we use `target` and `target` is always a single fully qualified type/function.

`@co.dap.nested` bechaves exactly like nested or inner classes or functions

---

Operators

Custom Operator Definition & Overloading

Operator functions are functions and cannot be declared loose at package scope. An operator function for a struct must be declared inside that struct's same-package companion unit.

The **first declared operand parameter** must have the matching struct type. A matching struct type in a later operand is insufficient. For a unary operator, the sole operand must have the matching struct type.

For infix operators, this makes the first parameter the ownership or left-operand type used for companion-unit lookup. For example, an operator in `Vector co.lang.unit` may define `Vector * scalar`, but it may not define `scalar * Vector` unless that operation is owned and provided by the scalar type's applicable implementation.

```
Employee co.lang.struct = {
    salary co.lang.int;
}

Employee co.lang.unit = {

    @co.dap.operator(symbol='+', mode=overload)
    add(a Employee, b Employee)->(Employee) = {
        this.return Employee{
            salary: a.salary + b.salary
        };
    }
}
```

The expression:

```
combined := first + second;
```

is resolved through the `Employee` companion unit.

```
Vector co.lang.struct = {
    x co.lang.float;
    y co.lang.float;
}

Vector co.lang.unit = {

    @co.dap.operator(
        symbol='∪',
        mode=define,
        fixity=infix,
        precedence=60,
        associativity=left,
        arity=binary,
        commutative=true,
        idempotent=true,
        identity="∅",
        foldable=true,
        vectorizable=false,
        distributes_over=['∩'],
        desugar="intrinsic:set_union"
    )
    union(left Vector, right Vector)->(Vector) = {
        ...
    }
}
```

Invalid placement and ownership:

```
@co.dap.operator(symbol='+', mode=overload)
add(a Employee, b Employee)->(Employee) = { ... }
// ❌ operator function cannot appear at package scope

Math co.lang.unit = {
    @co.dap.operator(symbol='+', mode=overload)
    add(a Employee, b Employee)->(Employee) = { ... }
    // ❌ Math is not the companion unit of Employee
}

Employee co.lang.unit = {
    @co.dap.operator(symbol='+', mode=overload)
    add(a co.lang.int, b Employee)->(Employee) = { ... }
    // ❌ first operand is not Employee; a later matching operand is insufficient
}
```

`mode=override` is not supported in the foreseeable future; the compiler reports an error.

fixity values: `infix`, `postfix`, `prefix`, `circumfix`, `postcircumfix`, `prescircumfix`, `mixfix`, `ternary`, `distfix`

Forward / Extern Declarations

Variables extern declaration

```
@co.dap.declare(extern)
someBool co.lang.bool;
```

Functions forward declaration

```
@co.dap.declare(forward)
getEmployee(id co.lang.int)->(somepack.Employee);

// or — @co.dap.declare is optional for functions
getEmployee(id co.lang.int)->(somepack.Employee);
```

Types external declaration

```
@co.dap.declare(extern)
Employee co.lang.struct;

// or — @co.dap.declare is optional for types
Employee co.lang.struct;
```

> For functions and types `@co.dap.declare` is optional. For variables it is required.

Functions

`folang` doesn't allow free flowing functions as UDTs they must be in package source file apart from that they must be enclosed in a kind of container call Unit `co.lang.unit`

Normal

```
General co.lang.unit = {

    fun1(k co.lang.int, b co.lang.char)->(co.lang.int, co.lang.char) = {
        // function body
    }
}
```

`folang` function can return multiple values

Default Parameters

```
fun1 (k co.lang.int, b co.lang.char = 10)->(co.lang.int, co.lang.char)={
}
```

Variadic Functions

Curried functions are not allowed to be variadic, and vice versa.

```
fun1 (k co.lang.int, ...b co.lang.char)->(co.lang.int, co.lang.char)={
}
```

Optional Parameters

```
fun1(k? co.lang.int)->()={
    if k.omitted{

    }else{

    }
}
```

Named Parameters

```
fun1(~k co.lang.int)->()={

}
```

Named Returns

```
doManythings(a co.lang.int, b co.lang.int->(&, meta={type=out}))->(r co.lang.int, e co.lang.exception)={}
```

Function Delegates

```
@co.dap.delegate someDelegate co.lang.delegate = (a co.lang.int, b co.lang.int) -> (co.lang.int, co.lang.int);
```

Function Chaining

```
fetchEmployee(empId co.lang.string)->(Employee)=>>empMod.getEmployee(this, empId);
```

Anonymous Functions

```
add := (a int, b int) -> (int) {
    this.return a + b;
};

res := (a int, b int) -> (int) {
    this.return a * b;
}(10, 20);
```

> for more details on functions please refer section Functions in Details

---

Functions in detail

Inline

```
Math co.lang.unit = {
    @co.dap.inline
    add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
        this.return a + b;
    }
}
```

Anonymous Functions

```
add := (a int, b int) -> (int) {
    this.return a + b;
};

res := (a int, b int) -> (int) {
    this.return a * b;
})(10, 20);
```

Lambda

Only allowed as an inline callback argument to collection operations (e.g. `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`). Using `|...|` anywhere else is a syntax/lint error.

```
// Syntax
|x, y| => x + y;

// Collection use — allowed
nums.map(|x| => x*x)
words.filter(|s| => s.len() > 3)
pairs.reduce(|acc, e| => acc + e, 0)
dict.map(|k, v| => v * 10)
list.sortBy(|a, b| => a.score - b.score)
```

Inner Function

```
myfun(a co.lang.int, b co.lang.int)->(co.lang.int)={
    p co.lang.int = 10;
    someother()->()={
        co.out.println(p);
    }
    someother();
    p = 20;
    someother();
}
```

As we have already seen the inner function capabilities achieved through either `@co.dap.local` or `@co.dap.nested`

Those two forms allow separation of inner function logic from main function so that improves readability. The above form is useful when inner functions are small and compact.

Curried

```
add(first co.lang.int)(second co.lang.int)->(co.lang.int)={
    this.return first + second

}
```

Closure

```
adder() -> ((co.lang.int) -> co.lang.int) ={
    sum co.lang.int = 0
    this.return  (x co.lang.int) -> (co.lang.int) = {
        sum += x
        this.return sum
    }
}
```

Functions Taking and Returning Functions

Syntax 1 — Inline signature

```
someFunction (r (co.lang.int, co.lang.int)->(co.lang.int))->((co.lang.int)->(co.lang.int))={}
```

Syntax 2 — Named type alias

```
someFArg co.lang.type = (co.lang.int, co.lang.int)->(co.lang.int)
someFRet co.lang.type = (co.lang.int)->(co.lang.int)

someFunction (r someFArg)->(someFRet)={}
```

Syntax 3 — Function objects

```
someFArg co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int)={
    this.return a + b;
}

someFRet co.lang.function = (a co.lang.int) -> (co.lang.int)={
    this.return a * 2;
}
```

Other ways to declare clsures/function objects and types/ curried functions

```
myobj co.lang.function = (a co.lang.int, b co.lang.int)->(co.lang.int)={
    this.return a + b;
}

add (a co.lang.int, b co.lang.int)->(co.lang.int){ this.return a + b; }
oObj co.lang.function = add;

funtype co.lang.type = (a co.lang.int, b co.lang.int)->(co.lang.int);

closure(factor int) => (x int) = x * factor;

curry(factor int)(val int) = factory * val;
```

Associated Functions

For a user-defined struct, associated functions must be declared inside the same-package companion unit whose name matches the struct. For more details on associated function please refer section Associated Functions in a Companion Unit

Some Restrictions on Special Functions

1. **Special functions**
   - Curried functions
   - Functions with named arguments
   - Functions with optional arguments
   - Functions with default arguments
   - Variadic functions
   - Functions that take or return functions or function types
   - Dynamically scoped and mixed-scoped functions
2. **Restrictions**
   - They cannot be overloaded.
   - They cannot be used as callbacks.
   - They cannot participate in Execution Models and Control Abstractions.

Scoping Rules for Functions

All functions in FoLang have a defined scope — the set of variables a function can access.

Default — Lexical / Static Scope

All of the following are **always** lexically scoped — this cannot be changed:

```
methods
inner methods
free functions
inner functions
target-local named functions
closures
lambdas
Generic Functions
Anonymous functions
Curried functions
```

Lexical scope means a function resolves names from its **declaration site**, not from the scope of its caller. Unit-level variables are forbidden, so a unit-scoped free function receives runtime values through parameters or introduces them locally. Inner functions may capture variables from the enclosing function scope.

```
ScopeExample co.lang.unit = {

    foo()->() = {
        x co.lang.int = 10;

        printX()->() = {
            co.out.println(x);  // ✅ x from the enclosing lexical scope
        }

        printX();
    }
}
```

Associated Functions — Additional Scope Options

Only associated functions support non-lexical scoping via annotations:

The examples below are members of the same-package `Employee` companion unit.

`@co.dap.lexicalscope` — default, explicit declaration

```
Employee co.lang.unit = {
    @co.dap.lexicalscope
    (emp Employee) process()->() = {
        co.out.println(emp.name);   // ✅ declaration scope
    }
}
```

`@co.dap.dynamicscope` — accesses caller's scope

```
Employee co.lang.unit = {
    @co.dap.dynamicscope
    (emp Employee) process()->() = {
        co.out.println(name);   // name comes from caller's scope
    }
}
```

`@co.dap.mixedscope` — accesses both scopes

```
Employee co.lang.unit = {
    // caller scope takes priority — shadows declaration scope on conflict
    @co.dap.mixedscope
    (emp Employee) process()->() = {
        co.out.println(name);   // caller scope takes priority
        co.out.println(emp.id); // falls back to declaration scope
    }
}
```

Callback Scope Inside Dynamically Scoped Associated Functions

A callback block or lambda does not independently select lexical, dynamic, or mixed scope. The associated function that executes the callback determines how the callback's free runtime names are resolved.

Callback parameters and declarations made inside the callback always belong to the callback's local scope. Names that are not callback parameters or callback-local declarations follow the executing associated function's scope policy.

```
nums.reduce(|acc, e| => {
    total.value += e;
    acc + e
}, 0)
```

For this example:

```
acc, e              -> callback parameters
temporary locals    -> callback-local context
total               -> reduce's dynamic caller context
co.* and imports    -> normal built-in/import resolution
```

The callback syntax remains uniform. `reduce` is dynamically scoped, so unresolved runtime names in the callback are resolved through the active caller-context chain. A lexically scoped associated function would instead resolve free runtime names through its declaration context.

Why Dynamic Scope Exists — `.do`, `.loop`, `.each`, and Collections

FoLang's control-flow model is built on dynamically scoped associated functions. The executing function supplies the scope policy, so blocks do not require separate capturing and non-capturing forms.

```
x     co.lang.int = 10;
total co.lang.int = 0;
arr   co.lang.int->([5]) = [1, 2, 3, 4, 5];

// .do reads and modifies the caller's x
(x > 5).do({
    x.value = 20;
    co.out.println(x);
})

// .loop modifies the caller's x
(x > 0).loop({
    x.value--;
})

// .each modifies the caller's total
arr.each(_, val).do({
    total.value += val;
})

// .filter, .map, and .reduce use the same dynamic caller context
nums.filter(|x| => x > 10)
nums.reduce(|acc, e| => {
    total.value += e;
    acc + e
}, 0)
```

The compiler does not create a separate capture description for each control block. It resolves names according to the scope mode of the associated function executing that block.

FoLang Control Flow Uses Dynamic Scope

```
no if/else keywords    -> .do / .otherwise  – dynamic scope
no for/while keywords  -> .loop             – dynamic scope
no foreach keywords    -> .each             – dynamic scope
no in keyword          -> .contains         – dynamic scope
no map/filter keywords -> .map / .filter     – dynamic scope

all control flow is implemented as associated functions
control associated functions are dynamically scoped
caller variables are accessible and mutable under the dynamic lookup rules
block syntax requires no separate capture mode
```

Dynamic and Mixed Lookup Order

For `@co.dap.dynamicscope`, runtime variable lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. built-in `co.*` names and imported declarations visible to the source file
5. compiler error if no compatible declaration is available
```

For `@co.dap.mixedscope`, lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. lexical declaration context
5. built-in `co.*` names and imported declarations visible to the source file
6. compiler error if no compatible declaration is available
```

Local declarations shadow caller declarations. Caller declarations shadow declarations from the lexical declaration context in mixed scope.

Types, annotations, imports, and other compile-time declarations continue to use ordinary static resolution. Dynamic lookup applies to runtime bindings, not to the identity of types or imported APIs.

Call-Site Validation of Dynamic Requirements

A dynamically or mixed-scoped associated function may reference a caller-provided runtime name that is not declared in its lexical context:

```
@co.dap.dynamicscope
(Employee) addToTotal(value co.lang.int)->() = {
    total.value += value;
}
```

At every statically known call site, the compiler validates that the active caller-context chain provides a definitely initialized binding named `total` with a compatible type and permitted mutability. A call is rejected when the required binding cannot be proven to exist.

Because non-lexically scoped functions are non-first-class and non-escaping, their call sites remain statically identifiable. This avoids runtime string-based name lookup and allows the compiler to represent dynamic requirements using context and symbol-table references.

Scope Rules Summary

Function Type	Lexical	Dynamic	Mixed
methods	✅ only	❌	❌
inner methods	✅ only	❌	❌
free functions	✅ only	❌	❌
inner functions	✅ only	❌	❌

Function Type	Lexical	Dynamic	Mixed
target-local named functions	✅ only	❌	❌
anonymous functions	✅ only	❌	❌
closures	✅ only	❌	❌
lambdas/callback blocks	determined by executing associated function	determined by executing associated function	determined by executing associated function
associated functions	✅ default	✅ opt-in	✅ opt-in
`.do` / `.loop` / `.each`	❌	✅ built-in	❌
`.map` / `.filter` / `.reduce`	❌	✅ built-in	❌

Additional Restrictions

- dynamically or mixed-scoped functions cannot be returned
- dynamically or mixed-scoped functions cannot be stored as values
- dynamically or mixed-scoped functions cannot be passed as ordinary callbacks
- dynamic or mixed caller contexts cannot cross thread or process boundaries
- dynamic or mixed execution cannot continue after the caller frame ends
- dynamic or mixed-scoped functions cannot participate in Execution Models and Control Abstractions (library type=advanced)
- dynamically or mixed-scoped functions cannot be curried
- named, optional, variadic, and default-parameter forms follow the same non-escaping and call-site-validation rules

Types

The three axes — each adds one new power:

Axis 1: Polymorphism (terms depend on types)

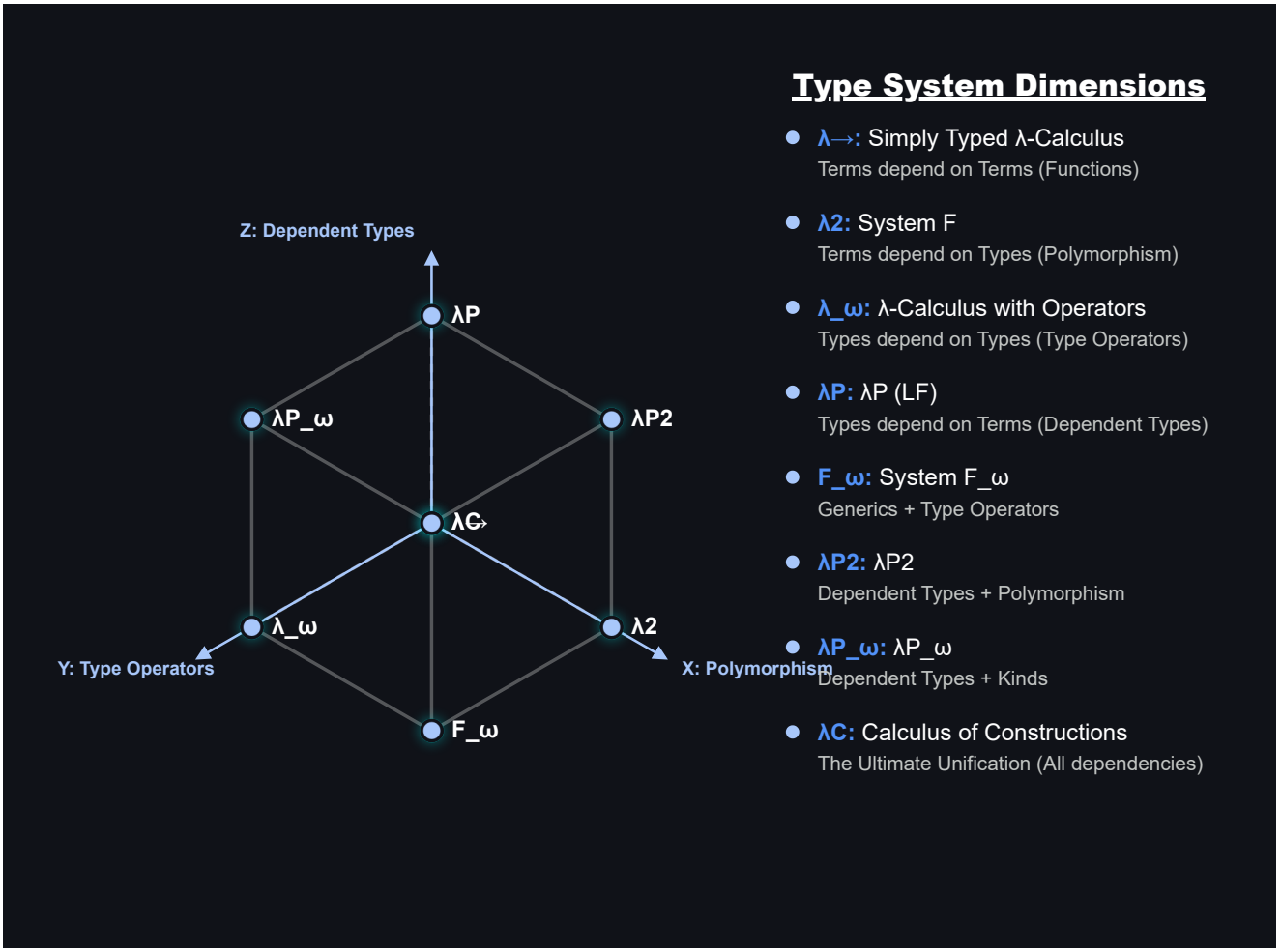
```
// "Give me a type, I'll give you a value"

// Without: write separate functions
identityInt(x int) → int
identityStr(x string) → string

// With: one function works for all types
identity(T)(x T) → T

// This is generics / parametric polymorphism
// System F, Java generics, your @co.dap.generic
```

Axis 2: Type operators (types depend on types)

```
// "Give me a type, I'll give you a type"

List(Int)     → List of Int       // type → type
Map(String, Int) → Map            // type → type → type
Option(T)     → Some(T) | None    // type constructor

// This is kinds / higher-kinded types
// Your folang: Option(T) co.lang.type = Some(T) | None()
```

Axis 3: Dependent types (types depend on values)

```
// "Give me a value, I'll give you a type"

Vector(3)      → array of exactly 3 elements
Matrix(2, 3)   → 2x3 matrix
NonZero(n)     → proof that n ≠ 0

// The type changes based on a runtime value
divide(a int, b NonZero(int)) → int
// compiler PROVES b can't be zero
```



Dependent Types

Type Constructors — Functions That Return Types

A type constructor is a function that takes a value or type and returns a type. The returned type depends on the input value — this is what makes it a dependent type.

```
// Vector — type constructor function
// takes  → co.lang.int (size)
// returns → co.lang.dependentType (a type)
Vector(n co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([n]);

// calling Vector(3) returns a TYPE at compile time
// that type is co.lang.int->([3])
v3 Vector(3) = [1, 2, 3];    // type is Vector(3)
v4 Vector(4) = [1, 2, 3, 4]; // type is Vector(4) — different type!

// Vector(3) ≠ Vector(4) — completely different types
// size is part of the type — compiler knows at compile time
```

More About Type

Name(T) co.lang.data = variants; → concrete ADT type-constructor definition → right-hand-side definition is mandatory

Name(T) co.lang.type; → abstract type-constructor requirement → permitted only inside a signature

Name(T) co.lang.type = ExistingType(T); → concrete type alias or signature-component binding → permitted in modules and ordinary type declarations

---

Type Constructor Is A Function

```
Vector        →  function (type constructor)
Vector(3)     →  function call → returns type co.lang.int->([3])
Vector(4)     →  function call → returns type co.lang.int->([4])

just like:
    add(1, 2)  →  returns a value  (3)
    Vector(3)  →  returns a type   (int[3])
```

Compiler Enforced Size Safety

```
// dot product — only valid for same size vectors
// compiler enforces this via dependent types
dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
    // n is same for both — compiler verified
}

v3 Vector(3) = [1, 2, 3];
v4 Vector(4) = [1, 2, 3, 4];

dotProduct(v3, v3)   // ✅ same type Vector(3)
dotProduct(v3, v4)   // ❌ compiler error — Vector(3) ≠ Vector(4)
```

Matrix — Two Parameter Type Constructor

```
// Matrix — takes rows and cols, returns dependent type
Matrix(r co.lang.int, c co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([r, c]);

m34 Matrix(3, 4) = [[1,2,3,4],[5,6,7,8],[9,10,11,12]];
m45 Matrix(4, 5) = ...;

// matrix multiply — cols of A must equal rows of B
// n must match — compiler verified
multiply(a Matrix(r, n), b Matrix(n, c))->(Matrix(r, c)) = {
    // compiler ensures dimensions are compatible
}

multiply(m34, m45)   // ✅ Matrix(3,4) × Matrix(4,5) = Matrix(3,5)
multiply(m34, m34)   // ❌ compiler error — 4 ≠ 3
```

Stack — Value and Type Parameter

```
// Stack — takes size and element type
Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) =
    T->([n]);

s Stack(10, co.lang.int)    = ...;  // stack of max 10 ints
t Stack(5,  co.lang.string) = ...;  // stack of max 5 strings
```

Type Is Value + Kind Combined

```
Vector(3):
    kind  = Vector    (what it is)
    value = 3         (how many)
    type  = Vector(3) (both together — the dependent type)

Vector(3) ≠ Vector(4)   ←  different types entirely
Vector(3) = Vector(3)   ←  same type
```

Connects to Type Constructor in Spec

```
// Option — type constructor for ADT
@co.dap.hokrt
Option(T) co.lang.data = Some(T) | None();

// Vector — type constructor for dependent type
Vector(n co.lang.int)->(co.lang.dependentType) =
    co.lang.int->([n]);

// same concept:
//   Option(T) takes a type  → returns ADT type
//   Vector(n) takes a value → returns dependent type
```

Simple Dependent Type

```
identity(x co.lang.int)->(x.type) = x
```

Compile-Time and Runtime Values in Type-Related Computation

FoLang distinguishes value-indexed dependent types, compile-time type computation, runtime type descriptors, and runtime values whose concrete types may differ. These mechanisms are related, but they are not interchangeable.

1. Value-Indexed Dependent Types

A dependent type may contain a value as part of its type identity. The value index may be a compile-time constant, a symbolic type-level value, or a runtime function parameter whose relationship to the result is tracked by the compiler.

```
readVector(n co.lang.int)->(Vector(n)) = {
    ...
}
```

The compiler does not need to know the concrete value of `n` while compiling `readVector`. It records the relationship:

```
input index = n
result type = Vector(n)
```

Likewise:

```
dotProduct(
    left  Vector(n),
    right Vector(n)
)->(co.lang.int) = {
    ...
}
```

requires both vectors to have the same value index. Dependent typing means that a type contains or is constrained by a value; it does not mean that an arbitrary runtime branch can silently change the static type of an already compiled variable.

---

2. Compile-Time Type Computation

A function may compute and return a type when it is guaranteed to execute during compilation.

```
@co.dap.comptime
@co.dap.eager
chooseType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return(co.lang.string)
        .otherwise.return(co.lang.bool);
}
```

The arguments must be compile-time evaluable when the result is used in a static type position:

```
Selected co.lang.type = chooseType(10);
value Selected = "Hello";
```

Invalid:

```
input := co.in.readInt();
Selected co.lang.type = chooseType(input);
// compiler error: `input` is not compile-time evaluable
```

Conceptually:

```
compile-time value
    -> compiler executes the type function
    -> one concrete static type is available before ordinary type checking completes
```

---

3. Runtime Type Descriptors

An ordinary function returning `co.lang.type` produces a runtime type descriptor when it is not executed at compile time.

```
selectType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return(co.lang.string)
        .otherwise.return(co.lang.bool);
}

selectedType co.lang.type = selectType(input);
```

Here, `selectedType` is a runtime object that describes a type. It may be used for reflection, runtime validation, dynamic loading, metadata inspection, or dynamic object creation where those capabilities are permitted.

A runtime type descriptor cannot ordinarily be used as the static type of a declaration:

```
value selectType(input);
// compiler error: runtime type descriptor used in a static type position
```

Conceptually:

```
runtime value
    -> runtime type-selection function
    -> co.lang.type descriptor object
```

A value that represents a type is not automatically a compile-time-resolved static type.

---

4. `@co.dap.typefromvalue`

`@co.dap.typefromvalue` derives a type from the type of a returned compile-time value. In the initial FoLang specification, it is permitted only for compile-time evaluation.

```
@co.dap.comptime
@co.dap.typefromvalue
inferType(value co.lang.int)->(co.lang.type) = {
    (value < 100)
        .return("Hello")
        .otherwise.return(co.const.true);
}
```

The compiler derives:

```
"Hello"       -> co.lang.string
co.const.true -> co.lang.bool
```

Every argument must be compile-time evaluable when the result is used in a type position. Runtime value-based type selection must instead use an explicit runtime representation.

---

5. Runtime Values with Different Concrete Types

When a runtime branch may produce unrelated concrete value types, the function must return one stable outer type. FoLang may use a tagged value, an ADT, `co.lang.dynamic` where permitted, or another explicitly packaged representation.

Using `co.lang.tag`:

```
selectValue(value co.lang.int)->(co.lang.tag) = {
    (value < 100)
        .return(co.lang.tag(co.lang.string, "Hello"))
        .otherwise.return(
            co.lang.tag(co.lang.bool, co.const.true)
        );
}
```

The outer static type is always `co.lang.tag`:

```
co.lang.tag
├── runtime type descriptor
└── value compatible with that descriptor
```

An ADT provides a more strongly typed alternative:

```
SelectedValue co.lang.data =
      StringValue(co.lang.string)
    | BoolValue(co.lang.bool);

selectValue(value co.lang.int)->(SelectedValue) = {
    (value < 100)
        .return(StringValue("Hello"))
        .otherwise.return(BoolValue(co.const.true));
}
```

Summary

```
Vector(n)
    -> value-indexed dependent type

@co.dap.comptime function returning co.lang.type
    -> compile-time type computation

ordinary runtime function returning co.lang.type
    -> runtime type descriptor

runtime branch returning unrelated value types
    -> tag, ADT, dynamic value, or another explicit package
```

A runtime type descriptor is a value that represents a type. It must not be confused with a statically resolved dependent type.

Indexer

Indexer functions for a struct are associated functions and must be declared inside the matching companion unit.

```
MyList co.lang.struct ={
    eles co.lang.int->([...]);
}

MyList co.lang.unit = {

    @co.dap.indexer(symbol="[]")
    (g MyList) get(index co.lang.int)->(co.lang.int) ={
        this.return g.eles[index]
    }

    @co.dap.indexer(symbol="[]=")
    (g MyList) set(index co.lang.int, value co.lang.int)->() ={
        g.eles[index] = value
    }
}

lst MyList;
co.out.println(lst[0]);
lst[1] = 22;
```

Generics

```
@co.dap.generic(
    at=runtime,
    type={
        T: {variance:invariant, bound=Number, kind:param, impredicative:false},
        R: {variance:invariant, bound=Number, kind:return}
    }
)
add(a T, b T)->(R) = { this.return a + b; }
```

Generic annotation fields:

- `at` — `runtime` | `compiletime` (acts like C++ templates)
- `refied` — `true` | `false`
- `where` — `usesite` | `callsite`

typename/type attributes:

Attribute	Values
variance	`covariant`, `invariant`, `contravariant`
bound	type to bind
kind	`param`, `result`, `var`, `arg`
default	default type
nullable	bool
inclusive	bool
impredicative	bool — when `true`, allows `T` to be instantiated with a `forall` type (v2)
typekind	`type`, `class`, `function`, `module`, `unit`, `package`
types	list of allowed types for constraints

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Syntax 1 — Inline signature

```
@co.dap.generic(type={T:{variance:invariant}})
someFunction(f (T, T)->(T), a T)->(T) = {}
```

Syntax 2 — Named type alias

```
@co.dap.generic(type={T:{variance:invariant}})
someFArg co.lang.type = (T, T)->(T)

someFunction(f someFArg, a T, b T)->(T) = {}
```

Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The callee decides what `T` is. Uses existing `forall`.

Syntax 1 — Inline signature

```
someFunction(f forall(T).(T, T)->(T))->(co.lang.int) = {
    this.return f(1, 2);
}
```

Syntax 2 — Named type alias

```
someFArg co.lang.type = forall(T).(T, T)->(T)

someFunction(f someFArg)->(co.lang.int) = {}
```

```
// Correct — Syntax 2 with co.lang.type
someFArg co.lang.type = forall(T).(T, T)->(T)

someFunction(f someFArg)->(co.lang.int) = {}
```

Returning Generic Functions

Rank-1 return

```
@co.dap.generic(type={T:{variance:invariant}})
makeAdder(a T)->((T)->(T)) = {
    this.return (b T)->(T) = { this.return a + b; };
}
```

Rank-2 return — returning a polymorphic function

```
makeIdentity()->( forall(T).(T)->(T) ) = {
    this.return forall(T).(x T)->(T) = { this.return x; };
}
```

Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 works naturally in FoLang via `forall` nesting. No new constructs needed.

Syntax 1 — Inline

```
// f takes a Rank-2 function as its argument — that is Rank-3
applyRank2(
    f (forall(T).(T, T)->(T)) -> (co.lang.int)
) -> (co.lang.int) = {
    this.return f(1, 1);
}
```

Syntax 2 — Named type aliases (cleaner)

```
rank2FnType  co.lang.type = forall(T).(T, T)->(T)
rank3ArgType co.lang.type = (rank2FnType) -> (co.lang.int)

applyRank2(f rank3ArgType) -> (co.lang.int) = {
    this.return f(1, 1);
}
```

Rank-3 return

```
makeRank2Consumer() -> ((forall(T).(T)->(T)) -> (co.lang.int)) = {
    this.return (f forall(T).(T)->(T)) -> (co.lang.int) = {
        this.return f(42);
    };
}
```

Impredicativity — Instantiating `T` with a `forall` Type

Impredicativity is when a type variable `T` in a generic is itself instantiated with a `forall` type. Example of what this means:

```
@co.dap.generic(type={T:{variance:invariant}})
box(x T) -> (Box(T)) = {}

// Impredicative call — T being set to forall(U).(U)->(U)
result := box(forall(U).(U)->(U));   // ❌ not legal without explicit opt-in
```

Most type systems reject this by default. FoLang takes an opt-in approach.

v1 Workaround — Option C: Wrapping with `co.lang.type`

Not true impredicativity but solves 90% of practical cases:

```
polyId co.lang.type = forall(U).(U)->(U)

// box takes co.lang.type — no impredicative unification needed
box(x co.lang.type) -> (Box(co.lang.type)) = {}

result := box(polyId);   // ✅ works — x is co.lang.type, not a forall type
```

v2 — Option A: `impredicative:true` in `@co.dap.generic`

Explicit opt-in via the existing annotation. The compiler only permits `forall` instantiation where declared:

```
@co.dap.generic(
    type={T:{variance:invariant, impredicative:true}}
)
box(x T) -> (Box(T)) = {}

polyId co.lang.type = forall(U).(U)->(U)
result := box(polyId);   // ✅ legal — impredicative:true explicitly opts in
```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
Rank-1 generic param (Syntax 1, 2, 3)	✅ Yes	Natural extension, no new concepts
Rank-1 generic return (Syntax 1, 2, 3)	✅ Yes	Same as above
Rank-2 param via `forall` (Syntax 1, 2)	✅ Yes	`co.lang.type` naturally holds polymorphic types; no new kind needed
Rank-2 param via Syntax 3 `co.lang.function`	❌ Compiler error	Function objects are concrete values; use `co.lang.type = forall(T).(T)->(T)` instead
Rank-2 return via `forall` (Syntax 1, 2)	✅ Yes	Same reasoning as param
Rank-3 via `forall` nesting (Syntax 1, 2)	✅ Yes	No new constructs — `forall` nesting works naturally
Rank-3 return	✅ Yes	Same reasoning as Rank-3 param

Scenario	Allow?	Notes
Rank-3 via Syntax 3 `co.lang.function`	❌ Compiler error	Same rule as Rank-2; function objects are concrete
Impredicative — v1 workaround (Option C)	✅ Yes	Wrap `forall` type in `co.lang.type`; solves 90% of real cases
Impredicative — true opt-in (Option A)	➡️ v2	`impredicative:true` in `@co.dap.generic`; explicit opt-in

Generic Types

```
@co.dap.generic(typename=T)
LinkedList co.lang.struct={
    value T
    next  LinkedList
    prev  LinkedList
}

k := LinkedList.new(co.lang.int); // when we call new it returns an object of type co.lang.uninit
actualList := k.init(); // this is what create a fully formed object of type class



@co.dap.generic(type={T:{typename}, R:{typename}})
Employee co.lang.class ={
    id   T
    name R

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee)={}
}

a := Employee.new(co.lang.int, co.lang.string);
b := a.init(1, "Rao");

Normally we need not use @@new and @@init it is special case only applicable for Generics
Normal conditions to create/instantiate object of class we just call init which internally call new
In specific cases as above we need to do two calls or use call chain like below

c:= Employee.new(co.lang.int,co.lang.string).init(1,"Rao");
```

Generics Inheritances and Types

```
This is in conceptual stage not supported.

A) Abstract vs concrete type members
B) Path-dependent types
    1. Type-level projection
    2. Path-dependent In folang how it would be
```

forall

What `forall` Is — and Is Not

`forall` is **not** a general-purpose generic declaration keyword. It is a **type-level expression only**, restricted to contexts where a polymorphic type must appear as an anonymous value inline — specifically Rank-2 and Rank-3 parameter and return positions, and `co.lang.type` aliases.

`@co.dap.generic` is the **one and only** way to declare generic functions, structs, classes, and other named things. `forall` at declaration level is a **compiler error**.

Where `forall` Is Allowed — Type Expression Form Only

`forall(T).` followed by an anonymous type body. The `.` is the syntactic signal that what follows is a type body, not a declaration name.

Pattern:

```
forall(T).  <anonymous type body>
```

```
// co.lang.type alias — naming a polymorphic type for reuse
someFArg co.lang.type = forall(T).(T, T)->(T)

// Rank-2 inline parameter — callee decides what T is
someFunction(f forall(T).(T)->(T)) -> (co.lang.int) = {}

// Rank-2 return type — returning a polymorphic function
makeIdentity() -> (forall(T).(T)->(T)) = {}

// Rank-3 inline parameter — f takes a Rank-2 function
applyRank2(f (forall(T).(T, T)->(T)) -> (co.lang.int)) -> (co.lang.int) = {}
```

Where `forall` Is Banned — Use `@co.dap.generic` Instead

```
// ❌ compiler error — forall at declaration level
forall(T) identity(x T)->(T) = {}

// ✅ correct
@co.dap.generic(type={T:{variance:invariant}})
identity(x T)->(T) = {}
```

```
// ❌ compiler error
forall(T) LinkedList co.lang.struct = { value T; next LinkedList; }

// ✅ correct
@co.dap.generic(typename=T)
LinkedList co.lang.struct = { value T; next LinkedList; }
```

```
// ❌ compiler error — Rank-1 generics belong to @co.dap.generic
forall(T) someFunction(f (T,T)->(T), a T)->(T) = {}

// ✅ correct
@co.dap.generic(type={T:{variance:invariant}})
someFunction(f (T,T)->(T), a T)->(T) = {}
```

Quick Reference

Form	Status	Context
`forall(T) name ...`	❌ Compiler error	Declaration level — use `@co.dap.generic` instead
`forall(T).(T)->(T)`	✅ Allowed	Type level only — Rank-2/3 param, return, `co.lang.type` alias

The rule in one sentence: `forall(T).` is a type constructor for anonymous polymorphic types; it is never a declaration keyword.

Templates

Typed

```
@co.dap.template
add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
    this.return a + b;
}
```

Untyped

```
@co.dap.template
add(a, b)->(co.lang.untyped) ={
    this.return a + b;
}
```

Annotations and Decorators

```
// Annotation — static object, can carry data


myAnnotation co.lang.object->(for=annotation) = {
    value   co.lang.string;
    enabled co.lang.bool;
}

// Decorator — function, transforms target, returns
@co.dap.decorator
myDecorator(target co.lang.function)->(co.lang.function) = { }

Note Directives and Pragmas are not allowed to create as they are language internals
```

Macros

```
// a. Basic macro
@co.dap.macro
say()->()={ this.return co.macro.quote({ println("Line 1") println("Line 2") }); }

// b. Escape assign
@co.dap.macro
yes_esc_assign()->(co.lang.untyped)={
    this.return co.macro.quote({
        co.macro.esc(y) = 42
        println("Inside macro: y = ", y)
    });
}

// c. Debug macro with gensym
@co.dap.macro
debug(expr)->(co.lang.untyped)={
    let tmp = co.macro.gensym(co.lang.var, "tmp")
    this.return co.macro.quote({
        tmp = co.macro.esc(expr)
        println("Result: ", tmp)
        tmp
    });
}

// d. if/else condition macro
@co.dap.macro(
    group = {items:["if","else"], chain:true},
    sugarform={forms:["if expr block"]},
    bind={vars:["x"]},
    isolate={vars:["temp", "index"]},
    gensym={prefix:"tmp_"},
    hygienic=true,
    argtransform={param:"body", wrap:"lambda", whentype:"block"},
    desugar={exprs:["if($cond) { $block }" => "if($cond,$block)"]},
    mode="inject"
)
if(condition expr, body block)->()={}

blockormacro co.lang.Kind = block | macro

@co.dap.macro(
    group={items:["if","else"], chain:true},
    sugarform={forms:["else block","else if"]},
    chainswith={macro:"if", position:"immediate", required:true},
    argtransform={param:"body", wrap:"lambda", whentype:"block"},
    standalone=false,
    desugar={exprs:[
        "else if($cond) { $block }" => "else(if($cond, $block))",
        "else { $elseblock }" => "else($elseblock)"
    ]},
)
else(body blockormacro)->()={}
```

Other macro utilities:

1. `@co.dap.compose(using=["base_if", "blockify"])`
2. `@co.dap.guard(expr="is_bool_expr(expr)")`
3. Quasiquote macros use `co.macro.quote` and `co.macro.unquote`

Execution Models and Control Abstractions (library type=advanced)

Foλang executes code sequentially by default. It also provides a uniform execution model for concurrency, parallelism, asynchronous execution, coroutines, continuations, scheduling, and structured task execution.

Developers express the intended execution semantics by applying annotations such as `@co.dap.thread`, `@co.dap.task`, or `@co.dap.process` to a method. When the method is submitted through facilities such as `co.cpca.submitToPool`, `co.cpca.submitThread`, or `co.cpca.submitToEventLoop`, the Fολang runtime selects and manages the appropriate execution mechanism.

Depending on the annotation, submission operation, runtime environment, and execution policy, Fολang may use a thread pool, virtual or green threads, an event loop, a dedicated operating-system thread, or a separate process. Communication operations such as sending and receiving values are also handled through the `co.cpca` package. Developers therefore describe the required execution behavior without directly managing the underlying threads, processes, pools, or event loops.

The `@co.dap.continuation` annotation enables continuation support for a function. An annotated function can use constructs provided by the `co.cpca` package to suspend execution, yield control or a value, preserve its execution state, and later resume from the suspension point.

Native Code (Library type system/ffi)

The `@co.dap.native` annotation enables access to the `co.native` package. Through this package, developers can write assembly and machine-level code using facilities such as `co.native.asm` and `co.native.inline`, providing low-level capabilities similar to those available in C++.

Native Functions

```
@co.dap.native
nativeMethod(a co.lang.int, b co.lang.int)->(co.lang.int) ={
    // native implementation
}
```

Dynamic Runtime (library type=dynamicvmrt)

The `@co.ddap.dynamicruntime` annotation enables full access to the `co.meta` package. Through this package, developers can use dynamic class and type loading, monkey patching, runtime reflection, instrumentation, eval-based code execution, and other advanced metaprogramming capabilities.

Variable Kinds Support

Kind	Where
Normal	All
Pointers	Library of type `system` and `ffi`
Arrays	All
References Heap, Lvalue, Rvalue	Library of type `system` and `ffi`
Addresses	Library of type `system` and `ffi`
Thunks	Library `application` or `system` or `ffi` or `advanced` or `dynamicvmrt`
Ranges	All
Slices	All

Builtin Data Types

Type	Kind
`co.lang.string`	
`co.lang.int`	
`co.lang.bit`	
`co.lang.double`	

Type	Kind
`co.lang.float`	
`co.lang.long`	
`co.lang.byte`	
`co.lang.char`	
`co.lang.any`	
`co.lang.dynamic`	
`co.lang.auto`	
`co.lang.infer`	
`co.lang.bool`	
`co.lang.void`	
`co.lang.data`	
`co.lang.value`	
`co.lang.typed`	
`co.lang.untyped`	
`co.mem.region`	
`co.lang.nothing`	
`co.lang.word`	
`co.lang.MatchBindings`	
`co.lang.tag`	
`co.lang.typevalue`	
`co.lang.uninit`	
`co.lang.Literal`	

Builtin Directives

Built-in Directives

Kind		
`PRAGMA`	"@co.pdap.compiler", "@co.pdap.scale"	
`DIRECTIVE`	"@co.ddap.movetotop", "@co.ddap.import", "@co.ddap.dynamicruntime", "@co.ddap.use", @co.ddap.parent", "@co.ddap.alias"	
`ANNOTATION`	"@co.dap.template", "@co.dap.macro","@co.dap.operator", "@co.dap.annotation", "@co.dap.library", "@co.dap.module", "@co.dap.pragma", "@co.dap.directive","@co.dap.native", "@co.dap.class", "@co.dap.static","@co.dap.instance", "@co.dap.object", "@co.dap.inline","@co.dap.ctfe", "@co.dap.friend", "@co.dap.sealed", "@co.dap.extension","@co.dap.override", "@co.dap.virtual", "@co.dap.abstract", "@co.dap.delegate", "@co.dap.dynamicscope","@co.dap.lexicalscope","@co.dap.staticscope""@co.dap.mixedscope", "@co.dap.typeclass","@co.dap.matcher", "@co.dap.constructor", "@co.dap.oops", "@co.dap.hokrt","@co.dap.hokrtl", "@co.dap.indexer", "@co.dap.generic", "@co.dap.comptime", "@co.dap.typefromvalue", "@co.dap.local", "@co.dap.private","@co.dap.public","@co.dap.package","@co.dap.protected","@co.dap.internal" ""@co.dap.export","@co.dap.eager", "@co.dap.lazy", "@co.dap.packed", "@co.dap.declare","@co.dap.simd", "@co.dap.reflection", "@co.dap.mop","@co.dap.nested","@co.dap.inner","@co.dap.declare"	//mop => meta object programming

Kind		
`DECORATOR`	"@co.dap.before", "@co.dap.after","@co.dap.around", "@co.fx.onErrExcept", "@co.fx.InvokeAlways","@co.fx.HandleEffect", "@co.dap.callback", "@co.dap.defer","@co.dap.continuation", "@co.dap.event", "@co.dap.scale", "@co.dap.distributed","@co.dap.concurrent", "@co.dap.parallel", "@co.dap.subroutine", "@co.dap.generator", "@co.dap.goroutine", "@co.dap.coroutine","@co.dap.async", "@co.dap.promise", "@co.dap.future", "@co.dap.thread", "@co.dap.task", "@co.dap.fiber", "@co.dap.process","@co.dap.spawn", "@co.dap.exec", "@co.dap.fork", "@co.dap.csp","@co.dap.actor", "@co.dap.synthetic", "@co.dap.bridge","@co.dap.greenlet", "@co.dap.channel", "@co.dap.callable", "@co.dap.iterator"	

Builtin Kinds

Kind	Purpose
`co.lang.type`	
`co.lang.struct`	
`co.lang.cstruct`	
`co.lang.realm`	similar to loader where symbols reside
`co.lang.loader`	class, type, functions loader where objects reside loader takes realm as parameter
`co.lang.class`	
`co.lang.interface`	
`co.lang.union`	
`co.lang.role`	
`co.lang.record`	
`co.lang.property`	
`co.lang.indexer`	
`co.lang.object`	
`co.lang.instance`	
`co.lang.matcher`	
`co.lang.trait`	
`co.lang.mixin`	
`co.lang.extension`	
`co.lang.delegate`	
`co.lang.typeclass`	
`co.lang.concept`	
`co.lang.typealias`	
`co.lang.module`	
`co.lang.unit`	named, non-instantiable function container; same-name struct unit acts as companion
`co.lang.macro`	
`co.lang.template`	
`co.lang.lambda`	
`co.lang.block`	
`co.lang.behavior`	

Kind	Purpose
`co.lang.package`	
`co.lang.signature`	
`co.lang.function`	
`co.lang.method`	
`co.lang.operator`	
`co.lang.namespace`	
`co.lang.stex`	
`co.lang.kind`	
`co.lang.level`	
`co.lang.order`	
`co.lang.rank`	
`co.lang.newtype`	
`co.lang.opaquetype`	
`co.lang.subtype`	
`co.lang.supertype`	
`co.lang.dependentType`	
`co.lang.refinementType`	
`co.lang.associatedtype`	
`co.lang.hkt`	higher kind type
`co.lang.hot`	higher orderr type
`co.lang.hrt`	higher rank type
`co.lang.data`	
`co.lang.enum`	
`co.lang.typetype`	
`co.lang.typekind`	
`co.lang.alias`	
`co.lang.value`	
`co.lang.just`	
`co.lang.nothing`	

Builtin Operators

Arithmetic operators

`+`, `-`, `*`, `/`, `%`, `**`, `++`, `--`

Logical operators

`&&`, `||`, `!`, `&`, `|`

Comparison operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

Other operators

`@`, `#`, `!`, `~`, `$`, `^`, `(`, `)`, `_`, `` ` ``, `?`, `{`, `[`, `]`, `}`, `\`, `:`, `;`, `"`, `'`, `=`, `.`, `?=`, `:=`, `::=`, `,`, `..`, `...`, `<..`, `..<`, `<..<`, `=>>`, `=>`, `->`, `<-`, `->>`, `<->`

Reserved words

`co`, `let`, `this`, `self (contextual keyword)`, `for`, `forall`, `fo (reserved word)`

Difference between `this` and `self`

- `this` is for instances and objects
- `self` is for classes
- `static` — no shortcut; can be on variable or classname
- Both `self` and `this` can access member variables

Builtin Methods

method	Responsibilit
to_str	
to_int	
to_float	
to_double	
classof	
typeof	
new	
prototype	
proto	
make	
objectof	
instanceof	
is	
as	
iskindof	
has	
hasown	
uses	
match	
matchall	
matchany	
matchnone	
matchtype	
case	
with	
print	
println	
printsp	

method	Responsibilit
echo	
contains	
cast	
to	
dummy	
clone	
of	
for	
when	
where	
then	
callback	
getAttr	
inject	
isinstance	
cast_to	
cast_from	
do	
map	
flatMap	
orElse	
filter	
fold	
recover	
peek	
loop	
istrue	
isfalse	
if	
elif	
else	
return	
otherwise	
each	
containsVal	
in	
iterate	
foreach	
decltype	deduce the type at compile time
replace	

method	Responsibilit
send	
receive	
submitToPool	
submitToEventLoop	

Builtin Packages

`co` — root (reserved word)

The only package provided by default.

Sub-package	Responsibility
`co.lang`	All data types and kinds
`co.sys`	file, concurrent, parallel, goto, invoke, bind, call, apply, settimeout, setinterval, scheduler, cron, event
`co.os`	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, dup2, close, readfd, writefd ,random
`co.meta`	ast, instrument, transform, augment, reflect, introspect, patch, inject, create, runtime(eval,proto,prototype,etc), realm
`co.core`	list, set, map, tree, trie, sort, search, array, pointer, ref, address, ptr, matrix, word
`co.native`	load, register, asm, inline, emit, ffi, spawnon[gpu,cpu,npu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]
`co.in`	read, readln
`co.out`	println, print
`co.regex`	stex, pattern, match, search
`co.crypto`	rsa, aes, hash, md5, rand, uuid, ssl, tls
`co.dap`	built-in decorators, annotations
`co.ddap`	built-in directives
`co.pdap`	built-in pragmas
`co.net`	tcp, udp, http
`co.const`	`true`, `false`, `none`
`co.encoding`	base64Encode, base64Decode, json, yml, bson
`co.utils`	makeImmutable, makeShared, copyOnWrite, toSnapshot — object behaviour policies
`co.dynamic`	dynamic capabilities
`co.runtime`	
`co.compiletime`	
`co.macro`	
`co.pattern`	
`co.control`	
`co.cpca`	concurrent, async, await, defer, lazy, parallel, process, thread,fiber, task, coroutine,continuation,cps, pool, channel ...
`co.hokrtl`	
`co.hokrt`	

FoLang Philosophy — Uniform Object Model

Core Principle

Everything in **FoLang** is an object. That is the reason for the name **FO** — **Functional Objects**.

FoLang gives the programmer one uniform object model across all types instead of forcing them to switch between separate type families for:

- ordinary values
- immutable values
- shared/concurrent values
- copy-on-write values
- literal values

The programmer writes against a single conceptual model and opts into the required object behaviour only when needed.

Uniform Object Principle

In FoLang, **everything is an object**.

Unless explicitly stated otherwise, the reference and mutation rules in this section apply to **managed FoLang objects**.
`co.lang.cstruct` remains an object in the language model, but it is an explicitly value-semantic ABI representation: assignment and parameter passing copy its value rather than a managed object reference.

This includes:

- scalar objects
- collection objects
- user-defined objects
- ADT objects
- function objects

Functions are objects in the same sense as all other values.
This is true for both **named functions** and **anonymous functions**.

FoLang does not introduce a separate semantic model for functions, scalars, UDTs, or ADTs.
They all follow the same core object principles:

- assignment of managed objects copies references
- assignment of `co.lang.cstruct` copies its value
- `==` compares values
- mutation is applied through the object
- behaviour policies such as immutability, shared, copy-on-write, and literal conversion apply uniformly where meaningful

So in FoLang, the programmer does not need one mental model for data objects and another for function values.
The language treats them under one consistent object model.

---

1. Default Object Model

Every value in FoLang is an object and is **mutable by default**.

This includes:

```
co.lang.int, co.lang.float, co.lang.string   → built-in scalar types
user-defined types (structs, classes, ADTs)  → user-defined objects
co.core.list, co.core.map, co.core.array     → built-in collections
```

Example:

```
positive_int co.lang.int = 10;
```

`positive_int` denotes an object.
Its current value is `10`.
By default, it is mutable.

---

2. Assignment, Aliasing, and Mutation

FoLang distinguishes three related ideas:

- **assignment**
- **aliasing**
- **mutation**

2.1 Assignment copies references

When one object variable is assigned to another, FoLang copies the **reference**, not the internal contents.

```
b := a
```

After this, `a` and `b` refer to the same object.

2.2 Mutation changes object contents

If two names refer to the same object, mutating through one name is visible through the other.

```
Employee co.lang.class = {
    Name co.lang.string
}

a Employee = { Name = "Kamesh" };
b := a;
c Employee = { Name = "Kamesh" };

a == b;   // true
a == c;   // true
b == c;   // true

b.Name = "Ramesh";   // changes a.Name too
c.Name = "Ramesh";   // does not change a.Name
```

Why:

- `a` and `b` are the **same object**
- `c` is a **different object** with the same earlier value

2.3 `==` means value equality

In FoLang, `==` compares **values**, not object identity.

That rule is uniform across all object kinds, including built-in types and user-defined types.

So:

```
a == b
```

means:

- compare contents deeply
- return `true` when the values match
- even if `a` and `b` are not the same object

2.4 Mutation accessors

The accessor used for mutation depends on the object type:

```
scalar (int, float, bool, string, char, byte)   → .value
struct field                                    → .fieldname
class member                                    → .membername
map entry                                       → .key
array / list element                            → .index
```

Examples:

```
count.value = 30
emp.name = "Rao"
marks.math = 95
nums.0 = 42  or nums[0] = 42 //both forms supported
```

2.5 Function-call behaviour

Function parameters introduce a **new local binding**.

So:

- rebinding a parameter does **not** affect the caller
- mutating the object through the parameter **does** affect the caller if both still refer to the same object

```
checkPositive(a co.lang.int)->(co.lang.bool) = {
    a = 20;         // local rebinding only — caller unchanged
    a.value = 30;   // object mutation — caller sees 30
}
```

If `positive_int` is passed to `checkPositive(a)`:

```
a = 20         → local rebinding only
a.value = 30   → mutates the passed object
```

---

3. Literal Objects

All literals in FoLang create **Literal Objects**.

```
"Kumar"   → Literal Object — string
42        → Literal Object — int
true      → Literal Object — bool
```

A Literal Object is an **anonymous object created from a literal expression**.

Literal Objects participate in value equality just like every other object in FoLang:

```
10 == 10   // true
```

In FoLang, `==` compares **values**, not object identity.

So two literal expressions with the same value compare equal, but that does not mean they are the same object.

Binding a Literal Object

When a literal is assigned to a named object, the name refers to the object created from that literal expression.

```
a co.lang.int = 10;
```

Here, `a` refers to the anonymous integer object created from literal `10`.

So:

```
a.value = 30;
```

mutates that same object, and `a` now has value `30`.

The important points are:

- literal expressions create anonymous objects

- literal-created objects are still ordinary objects unless another policy is applied
- `==` compares values, not identity
- once bound to a name, a literal-created object behaves like any other normal mutable object
- immutability applies only when explicitly requested, for example through `makeImmutable(...)`

Why Literal Objects Cannot Be Mutated Directly

Mutation can occur in two ways:

1. Through rebinding

   a co.lang.int = 10; a = 20;

   The above statement is valid because a is valid identifier and named handle to `co.lang.int` type.

   10 = 20; // ❌ invalid

   The above statement is invalid because 10 is not a valid identifier and if 10 in literal object type there is no named handle.

   The rebinding happens through named handles which are valid identifiers of `folang`

2. Through a property or method

   An object may also be mutated through one of its mutable properties or methods. a co.lang.int = 10; a.value = 20;

   A literal object cannot be mutated directly because it does not provide properties/methods that can be accessed.

   10.value = 20; // ❌ invalid

```
a co.lang.int = 10;
a.value = 30;
```

is valid after binding, a bare literal such as `10` cannot be mutated directly because there is no name or handle through which to perform mutation.

Not the Same as `toSnapshot(...)`

A literal object created by a literal expression is **not** the same thing as the internal snapshot representation produced by `co.utils.toSnapshot(...)`.

- literal expression → anonymous object
- `toSnapshot(x)` → internal snapshot representation used to reconstruct a fresh local object later

These are related concepts, but they are not the same mechanism.

Summary

- literal objects are real objects
- literal expressions create anonymous objects
- identical literals compare equal by value
- identical literals are not automatically the same object
- literal-created objects are mutable by default once bound to a handle
- only `makeImmutable(...)` makes an object immutable

---

4. Object Behaviour Policies

Any object can be given a behaviour policy using `co.utils.*`.

All four policy calls are **in-place transformations**:

- the object itself changes behaviour kind
- there is no wrapper object
- there is no alternate binding to capture
- the original name now refers to the transformed object

All policies are **deep by default**.
They flow through nested structs, members, collection elements, and all reachable objects in the graph unless the specification later states otherwise.

4.1 Immutable

```
co.utils.makeImmutable(positive_int)
```

After this call the object itself is immutable.

This is an in-place transformation, not a wrapper.

Any attempt to mutate the object or any reachable object beneath it is a **compiler error** where detectable statically, and a **runtime error** otherwise.

```
positvie_int co.lang.int = 10;
positive_int.value = 30   // ❌ compiler error
positive_int = 30         // ❌ compiler error
```

For nested objects:

```
Employee co.lang.struct = { address Address; }
Address co.lang.Address = {city co.lang.string; state co.lang.string; lane co.lang.string;pin co.lang.string;}


emp Employee = Employee{
    address = Address{
            city: "Pune";

    }
}
co.utils.makeImmutable(emp);

emp = Employee{
    address=Address{
            city: "ABC";

    }
} // ❌ compiler error


emp.address = newAddress              // ❌ compiler error
emp.address.city.value = "Mumbai"     // ❌ compiler error
```

Immutability is deep and total.

4.2 Value Immutable

```
positive_int co.lang.int = 20;

co.utils.makeValueImmutable(positive_int);

positive_int.value = 30; // ❌ current object is immutable
positive_int = 30;       // ✅ binding may reference a new object

co.utils.makeValueImmutable(emp);

emp.address.city = "ABC";       // ❌
emp.address.city.value = "ABC"; // ❌

emp = Employee{
    address= Address{
        city: "ABC";
    }
}; // ✅
```

Difference between Immutable and Immutable Value

```
    makeValueImmutable(x)
        └── current object graph cannot change

    makeImmutable(x)
        ├── current object graph cannot change
        └── binding cannot be reassigned
```

Table

Operation	Binding	Current value/object graph
`makeValueImmutable(x)`	Mutable	Immutable
`makeImmutable(x)`	Immutable	Immutable

4.3 Shared

```
co.utils.makeShared(positive_int)
```

A Shared Object is safe for concurrent and multi-threaded use.

The programmer continues to use the same object model — same accessors, same syntax — while FoLang guarantees the required safety properties internally.

Shared behaviour is deep:

- all reachable objects within the shared object are also shared

Note on Analogies

Comparisons to things like Java's `AtomicInteger` or `ConcurrentHashMap` are **analogies only**.

They may help explain the concept, but they are **not part of the formal language contract**.

FoLang specifies the behavioural guarantee, not the exact runtime implementation.

A good explanatory statement is:

> A shared `co.lang.int` may be thought of similarly to an atomic integer, and a shared map may be thought of similarly to a concurrent map. These comparisons are explanatory only. FoLang does not require any particular internal runtime representation.

4.4 CopyOnWrite

```
co.utils.copyOnWrite(positive_int)
```

A CopyOnWrite Object passes by reference like a normal object.

The copy is deferred.
It is created only when mutation is attempted in a context that must not affect the original source object.

```
process(a co.lang.int)->() = {
    a.value = 99;   // deep copy is made here — caller's object unchanged
}

co.utils.copyOnWrite(positive_int)
process(positive_int)

// positive_int still holds its old value
```

Copy-on-write is deep.
When a copy is made, the entire reachable object graph is copied.

Cyclic references must be handled by the runtime/compiler's structural clone logic with proper identity tracking.

4.5 toSnapshot

```
co.utils.toSnapshot(positive_int)
```

`toSnapshot` converts an object into a **snapshot representation** — a value descriptor, not a live object. The snapshot representation itself cannot be mutated.

When passed to a function, the compiler/runtime uses the snapshot representation to construct a fresh independent local variable bound to the parameter name. That local is a normal mutable object with no shared identity with the original. All of this happens automatically — the developer writes only `co.utils.toSnapshot(positive_int)`.

```
process(a co.lang.int)->() = {
    a.value = 99;   // mutates the fresh local — positive_int completely unaffected
}

process(co.utils.toSnapshot(positive_int))

// positive_int unchanged
```

The flow:

```
positive_int
    ↓  co.utils.toSnapshot(positive_int)
    snapshot representation         ←  value descriptor — immutable by nature, not by policy
    ↓  passed to function
    compiler constructs fresh local 'a' from the snapshot representation
    'a' is a new independent normal object — mutations never reach positive_int
```

`toSnapshot` is deep — the snapshot representation covers the entire reachable object graph.

---

5. Policy Summary

Object Kind	Mutation allowed	Caller sees mutation	Thread safe	Copy on write	Deep
Literal expression object	❌ directly in source — no handle	—	value-like use	—	✅
Normal (default)	✅ via accessor	✅	❌	❌	—
Immutable	❌	—	✅	—	✅
Shared	✅ via accessor	✅	✅	❌	✅
CopyOnWrite	✅ on own copy	❌	✅	✅	✅
toSnapshot result	✅ on reconstructed local object	❌	independent snapshot	—	✅

---

6. No Type Fragmentation

FoLang deliberately avoids special types for mutability or concurrency concerns.

There is no need for separate public type families such as:

```
ImmutableInt
SharedMap
CopyOnWriteList
AtomicInt
ConcurrentMap
```

Instead, FoLang keeps the type model uniform and applies behaviour policies through `co.utils.*`.

In many ecosystems, a programmer must constantly choose between:

```
normal integer      vs atomic integer
normal map          vs concurrent map
mutable value       vs immutable wrapper
raw structure       vs copy-on-write structure
```

FoLang instead aims to provide:

- one object model
- one programming style
- one mental model

while still allowing the programmer to opt into immutability, sharing, copy-on-write, or literal conversion when needed.

---

7. What Still Needs Precision

The philosophy is sound, but the formal specification still needs to define these precisely:

```
aliasing behaviour                    → when two names refer to the same object
rebinding vs mutation interaction     → edge-case coverage
deep policy propagation rules         → exactly which objects are reachable
cyclic reference handling in COW      → identity-tracking specification
visibility of mutation across calls   → formal function-call model
identity vs value equality            → what operators or built-ins expose identity
policy stacking                       → can an object be both shared and COW?
                                          can an object be both shared and immutable?
```

---

8. Formal Philosophy Statement

> All managed FoLang objects use reference semantics by default. `co.lang.cstruct` is an explicitly value-semantic ABI representation and is an exception to managed-object reference semantics.
> In FoLang, everything is an object and managed objects are mutable by default.
> Assignment of managed objects copies references, `co.lang.cstruct` assignment copies values, and `==` compares values deeply.
> Developers may opt into immutability, shared behaviour, copy-on-write behaviour, or literal conversion depending on their needs.
> All behaviour policies are deep — they apply to the entire reachable object graph unless stated otherwise by the formal specification.
> This policy model is uniform across managed types, so programmers do not need separate type families for ordinary, immutable, concurrent, or snapshot-oriented use.
> Familiar analogies such as atomic integers or concurrent maps may help explain the design, but they are not part of the formal implementation contract.
